Small Neural Policies are Enough: Learning-Guided Simulated Annealing for Large-Scale Capacitated Vehicle Routing

Anonymous submission

Abstract

The Capacitated Vehicle Routing Problem (CVRP) is a fundamental NP-hard optimization problem with numerous real-world applications. We propose a substantially improved Learning-Guided Simulated Annealing (LG-SA) model that combines the robustness of classical simulated annealing (SA) with the adaptive guidance of reinforcement learning (RL). The learned policy is parameterized by only two small multi-layer perceptrons operating on a node-centric solution representation whose dimensionality is independent of the problem size. As a result, thousands of instances can be processed efficiently in parallel on a single GPU. The framework combines an enriched node representation, a stronger critic, stabilized Proximal Policy Optimization (PPO), curriculum learning, and logit clipping for improved exploration. The framework is systematically optimized through a large-scale seed-paired study evaluating a broad range of design choices. The resulting model significantly improves the original LG-SA, reducing the gap to the reference metaheuristic HGS to 2.30% using a budget of 10^5 annealing steps. Moreover, the model solves a benchmark of $10{,}000$ instances in 29 minutes on a single GPU, matching or outperforming the leading learning-based improvement methods NLNS and LIH at five to ten times lower computational cost. Finally, a single model trained on 100-customer instances generalizes without retraining to instances containing up to $10,000$ customers with near-linear runtime scaling. Overall, these results demonstrate that carefully designed lightweight hybrid methods can achieve a substantially better quality/computation trade-off than state-of-the-art learning-based improvement methods.

1 Introduction

The Capacitated Vehicle Routing Problem (CVRP) asks for a minimum-cost set of routes serving all customers from a single depot under a vehicle capacity constraint. Central to combinatorial optimization since Dantzig and Ramser (1959) and NP-hard, it is out of reach of exact methods at realistic scales (Toth and Vigo 2002), so practical solvers are heuristic: a constructive rule (Clarke and Wright 1964) produces a first solution that local search and metaheuristics refine. Simulated annealing (SA) (Kirkpatrick, Gelatt, and Vecchi 1983) is among the simplest of these: a random proposal is accepted by the Metropolis criterion under a decreasing temperature. SA remains attractive for its $\mathcal{O}(1)$ per-step cost. The state of the art, in contrast, is held by heavily engineered metaheuristics (Vidal 2022; Helsgaun 2017; Accorsi and Vigo 2021), which reach near-optimal solutions but are inherently sequential, CPU-bound, and demand substantial per-instance tuning (Liu et al. 2023).

Neural combinatorial optimization instead learns a solver from data (Wu et al. 2024). *Construction* methods build a solution autoregressively (Nazari et al. 2018; Kool, van Hoof, and Welling 2019; Kwon et al. 2021); *improvement* methods start from a feasible solution and learn which local-search move to apply next (Chen and Tian 2019; Wu et al. 2022; Ma et al. 2021; Ma, Cao, and Chee 2023). Both families reach strong solution quality, but they share a structural cost: a heavyweight neural encoder is re-evaluated at every one of thousands of construction or search steps, which dominates the runtime and caps both the step budget and instance size. Their reported quality is therefore bought with computation. On the standard benchmark of $10{,}000$ instances with 100 customers, none of the learned methods surveyed in Table 3 completes the set in less than two hours.

This work follows a third, less-explored line: hybrid methods that keep a classical metaheuristic intact and learn only one of its components. Neural SA (Correia, Worrall, and Bondesan 2023) replaces the uniform proposal distribution of SA by a small policy trained by reinforcement learning, leaving the Metropolis criterion and the cooling schedule untouched. Learning-Guided Simulated Annealing (LG-SA) (Andretti, Cabessa, and Strozecki 2026) transposes the idea to the CVRP with a policy whose action is a pair of nodes, together with action masking. The appeal of the hybrid is that the model does not solve the problem: it merely guides an algorithm that is already correct and fast, and can therefore remain very small. Our guiding philosophy pushes this logic to its limit: training may be elaborate, but inference must stay simple and fast. The policy is accordingly restricted to two multi-layer perceptrons with a single hidden layer of 32 neurons, operating on a node-centric representation whose dimensionality is independent of the instance size, so that for an instance with N customers an annealing step reduces to $\mathcal{O}(N)$ batched tensor operations and thousands of instances anneal in parallel on a single GPU.

Within this deliberately fixed inference budget, we show that the room left for improvement lies entirely in what the policy *sees* and how it is *trained*, not in how large or complex it is.

Contributions.

- *A richer state, at constant model size.* We extend the node-centric representation with descriptors of a node's *neighborhood* (local density, insertion detour, distance to the route centroid), without adding a single layer or neuron. This is the largest contributor to the final gain, and every feature is constrained to be computable in constant time per node by batched, vectorized tensor operations, so throughput is preserved
- *An improved learning process.* We introduce a multi-statistic critic, a tuned and KL-regularized PPO, and logit clipping on the proposal distribution, which prevents it from collapsing: this both stabilizes training and preserves exploration at inference. A curriculum then bridges training and deployment by starting episodes from increasingly-improved solutions, so that few-hundred-step rollouts prepare the policy for the long annealing horizons used at inference.
- *Systematic experimental design.* Every design decision, from the representation and the architecture to the reward, the annealing parameters, the initialization, the optimization and the curriculum, is selected by a seed-paired sequential study over 18 design dimensions and 93 candidate configurations, with significance judged against a per-batch noise floor rather than by convention. We report the confirmed ties and the failed extensions alongside the gains: in particular, sparse rewards, heavier actor/critic architectures, and global-context features bring no significant gain, reinforcing that the leverage lies in the state and the training, not in model capacity.
- *Results and generalization.* At $N = 100$, the gap to HGS drops from 11.55% for the original LG-SA to 3.98% at 10^4 annealing steps, at an unchanged runtime of under three minutes for the whole $10{,}000$-instance benchmark, and to 2.30% at 10^5 steps in 29 minutes, matching NLNS and outperforming LIH with five to ten times less computation. A single model further transfers without retraining to instances of up to $10{,}000$ customers and to a different instance distribution, with near-linear runtime scaling.

2 Related Works

Classical metaheuristics. The strongest CVRP solvers are heavily engineered metaheuristics. LKH-3 (Helsgaun 2017) extends the Lin–Kernighan k-opt method, designed for the TSP, to constrained routing through penalty functions. HGS (Vidal 2022) is a hybrid genetic search: a population of solutions evolves by crossover, each offspring is refined by local search, and an explicit diversity management balances quality against exploration. Combined with the SWAP* neighborhood, it is the reference on uniform CVRP. FILO (Accorsi and Vigo 2021) is an iterated local search that keeps each ruin-and-recreate perturbation localized, and scales to instances with tens of thousands of customers. General-purpose solvers such as OR-Tools (Furnon and Perron 2024) are dominated by all three. All these methods are sequential, CPU-bound, and performance rests on per-instance tuning (Liu et al. 2023).

Learning to construct. *Construction* methods build a solution sequentially: a learned policy appends one customer at a time, conditioned on the partial routes, until all customers are served. Pointer networks (Vinyals, Fortunato, and Jaitly 2015; Nazari et al. 2018) pioneered the approach, later strengthened by the Transformer-based Attention Model (Kool, van Hoof, and Welling 2019) and POMO (Kwon et al. 2021), which exploits solution symmetries. A single pass is rarely sufficient, hence the reported quality comes from search procedures added at inference: sampling many candidate solutions, simulation-guided beam search (Choo et al. 2022), or per-instance fine-tuning of the policy (Hottung, Kwon, and Tierney 2021; Kim, Park, and Kim 2021). A parallel line targets generalization to large instances (Drakulic et al. 2023; Luo et al. 2023).

Construction methods build a solution autoregressively, from pointer networks (Vinyals, Fortunato, and Jaitly 2015; Nazari et al. 2018) to the Transformer-based Attention Model (Kool, van Hoof, and Welling 2019) and POMO (Kwon et al. 2021), which exploits solution symmetries. A single pass being rarely near-optimal, most of their quality now comes from test-time search stacked on the policy (Kim, Park, and Kim 2021; Hottung, Kwon, and Tierney 2021; Choo et al. 2022), while a parallel line targets large-scale generalization (Drakulic et al. 2023; Luo et al. 2023).

Learning to improve. *Improvement* methods formulates the search as an MDP: starting from a feasible solution, a policy selects the next local-search move. They differ in what is learned. NeuRewriter (Chen and Tian 2019) and L2I (Lu, Zhang, and Yang 2020) learn to choose among rewriting rules or operators. LIH (Wu et al. 2022), DACT (Ma et al. 2021) and NeuOpt (Ma, Cao, and Chee 2023) instead fix the operator and learn its *arguments*, a node pair for the first two, a k-opt move for the third. The latter two hold the best learned local-search results on uniform CVRP, through separate node- and position-embedding streams (DACT) and a scheme that guides exploration through infeasible regions (NeuOpt). NLNS (Hottung and Tierney 2022) learns the repair operator of a large-neighborhood search (García-Torres et al. 2022; Bui and Mai 2023); others learn only a representation steering a non-neural search (Hottung, Bhandari, and Tierney 2021; Kool et al. 2022). These methods are based on a heavyweight encoder re-evaluated at every one of thousands of steps, which dominates the runtime and caps both the step budget and the instance size.

Learning-guided metaheuristics. Closest to our work, a few methods leave a classical metaheuristic intact and learn a single component of it. Neural SA (Correia, Worrall, and Bondesan 2023) replaces the proposal distribution of simulated annealing with a small equivariant policy trained by PPO, while the Metropolis criterion and the cooling schedule are untouched. It was not evaluated on constrained routing, where capacity renders most moves infeasible. LG-SA (Andretti, Cabessa, and Strozecki 2026) brings the idea to the

CVRP with a policy whose action is a pair of nodes—the two customers on which the local-search move operates—together with action masking. We build on that framework. More broadly, hybridizing learning with mature metaheuristics is an active recent direction, for instance RL-finetuned operators for hybrid genetic search (Zhu, Zhang, and Cao 2025).

3 CVRP

The *Capacitated Vehicle Routing Problem (CVRP)* is defined on a complete undirected graph $G = (V, E)$, where $V = \{0\} \cup \mathcal{C}$ and $E = V \times V$ denote the sets of nodes and edges, respectively. Node 0 corresponds to the central depot, and $\mathcal{C} = \{1, \dots, N\}$ represents the set of N customers.

Each edge $(i, j) \in E$ is associated with a travel cost $c_{ij} = \|\mathbf{p}_i - \mathbf{p}_j\|_2$, the Euclidean distance between the 2D coordinates $\mathbf{p}_i, \mathbf{p}_j \in [0, 1]^2$, satisfying $c_{ij} = c_{ji}$. Each customer $i \in \mathcal{C}$ has a non-negative demand q_i, while the depot has zero demand ($q_0 = 0$). An unlimited fleet of identical vehicles, each with uniform capacity Q, is stationed at the depot.

A route $R_k = (0, v_1, v_2, \dots, v_p, 0)$ is a simple cycle starting and ending at the depot, where $\{v_1, \dots, v_p\} \subseteq \mathcal{C}$ denotes the subset of customers served by vehicle k. A *solution* $s = \{R_1, \dots, R_m\}$ consists of a set of m routes, where the number of routes m is unconstrained. A solution s is *feasible* if and only if it satisfies the following conditions:

1. Each customer is visited exactly once by a single vehicle,

$$\sum_{k=1}^{m} \mathbf{1}_{\{i \in R_k\}} = 1, \quad \forall i \in \mathcal{C}.$$

2. For every route R_k, the total demand of its customers does not exceed the vehicle capacity Q, i.e.,

$$\sum_{i \in R_k \setminus \{0\}} q_i \leq Q, \quad \forall k = 1, \dots, m.$$

The set of *feasible solutions* is denoted by $\mathcal{F}$.

Let $x_{ij}(s)$ be a binary indicator variable with $x_{ij}(s) = 1$ if edge (i, j) is traversed in any route R_k of s, and $x_{ij}(s) = 0$ otherwise. The objective of the CVRP is to find a feasible solution $s^* \in \mathcal{F}$ that minimizes the total travel cost $C(s)$:

$$s^* = \arg\min_{s \in \mathcal{F}} \Big\{ C(s) = \sum_{i \in V} \sum_{j \in V,\, i<j} c_{ij}\, x_{ij}(s) \Big\}.$$

4 Model

We introduce an improved version of the *Learning-Guided Simulated Annealing (LG-SA)* algorithm (Andretti, Cabessa, and Strozecki 2026; Correia, Worrall, and Bondesan 2023), a hybrid framework combining Simulated Annealing (SA) with a lightweight neural policy trained with PPO. Our model is tailored to the CVRP by means of a dedicated state representation, an efficient two-stage policy, and an optimized RL training procedure. The resulting algorithm preserves the computational efficiency of classical simulated annealing while significantly improving search effectiveness.

Overview. LG-SA starts from an initial feasible solution $s_0 \in \mathcal{F}$ and follows the standard simulated annealing search procedure. At iteration t, the current solution s_t is encoded into a node-centric representation $\bar{s}_t$ (described next). Then, an action a_t, which is a pair of nodes (i, j), is sampled from the learned policy π_θ

$$a_t = (i, j) \sim \pi_\theta(\cdot \mid \bar{s}_t)$$

and passed to a neighborhood operator H, which transforms the current solution s_t into the candidate solution

$$s'_t = H(s_t, a_t)\,.$$

This solution is accepted as the next solution s_{t+1} according to the classical Metropolis criterion. Consequently, LG-SA learns only the proposal distribution π_θ, while the neighborhood operator, Metropolis acceptance criterion, and cooling schedule remain those of standard SA. The overall search procedure is summarized in Algorithm 1.

Algorithm 1 LG-SA

```
Initialize solution s_0 and temperature T_0
for t = 0, 1, ... until the stopping criterion is met do
    Encode current solution s_t as s̄_t
    Sample action a_t ~ π_θ(· | s̄_t)
    Generate candidate solution s'_t ← H(s_t, a_t)
    Accept s'_t as s_{t+1} according to the Metropolis criterion
    Update the temperature
end for
```

Solution encoding. A solution s_t consists of an ordered sequence of customer nodes separated by depot visits. At each iteration t, LG-SA computes a node-centric representation $\bar{s}_t = \{\mathbf{x}_k\}_{k \in \mathcal{C}}$ of s_t, where each $\mathbf{x}_k$ is a feature vector encoding both static and dynamic information associated with customer k in the current solution and search state. The encoded representation $\bar{s}_t$ constitutes the input to the neural policy π_θ.

Our feature representation combines static geometric information, dynamic descriptors of each node's position within the current solution (e.g., predecessor and successor, local detour cost, and distance to the route centroid), route capacity statistics, and search meta-features such as the current temperature and remaining search budget. Unlike conventional geometric encodings, it also includes quality-related signals that explicitly characterize the local quality of the current solution around each node.

Since node features have a fixed dimensionality independent of the instance size, the same policy naturally generalizes to CVRP instances of arbitrary size and evaluates all nodes in a batch through a single vectorized tensor computation. The complete feature definitions are provided in Appendix A.1.

Two-stage policy. For every solution representation $\bar{s}_t = \{\mathbf{x}_k\}_{k \in \mathcal{C}}$, the learned policy defines a distribution over node pairs, from which an action $a_t = (i, j)$ is sampled:

$$a_t = (i, j) \sim \pi_\theta(\cdot \mid \bar{s}_t)\,.$$

Directly parameterizing this joint distribution over all $\mathcal{O}(N^2)$ node pairs is computationally expensive and unnecessary. We therefore factorize the policy as

$$\pi_\theta\big(a_t = (i,j) \mid \bar{s}_t\big) = \pi_\theta^{(1)}(i \mid \bar{s}_t) \cdot \pi_\theta^{(2)}(j \mid i, \bar{s}_t)\,,$$

where the two conditional distributions are parameterized by two small multi-layer perceptrons (MLPs). The first network f_θ receives all node features $\{\mathbf{x}_k\}_{k\in\mathcal{C}}$ in parallel, computes a score for each node independently, and samples the first node i. Conditioned on this selection, the second network g_θ receives the pair representations $\{[\mathbf{x}_i, \mathbf{x}_k]\}_{k\in\mathcal{C}}$ in parallel also, computes a score for each candidate partner, and samples the second node j. The neural networks f_θ and g_θ are illustrated in Figure 2 (Appendix A.2).

Hence, each action-sampling step requires only $\mathcal{O}(N)$ network evaluations, instead of explicitly scoring all $\mathcal{O}(N^2)$ node pairs. Since both MLPs are lightweight and process all nodes in parallel, each sampling step can be executed efficiently on a GPU.

Neighborhood operator. The action $a_t = (i,j)$ sampled from $\pi_\theta(\cdot \mid \bar{s}_t)$ is interpreted by a neighborhood operator H, which transforms the current solution s_t into the candidate solution

$$s'_t = H(s_t, a_t)$$

by applying the corresponding local-search move.

We evaluate three standard pairwise neighborhood operators: (i) *Swap*, (ii) *Insertion*, and (iii) *2-opt/2-opt**, which respectively exchange two nodes, relocate a node, and reconnect route segments, as illustrated in Figure 3 (Appendix A). The neighborhood operator is fixed for a given model. Consequently, the policy learns a proposal distribution specialized for the selected operator.

Reinforcement learning. The LG-SA search process is formulated as a Markov Decision Process (MDP). The encoded solution $\bar{s}_t$ constitutes the state, the sampled node pair $a_t = (i,j)$ the action, and the transition combines the neighborhood operator with the stochastic Metropolis acceptance criterion. The reward is derived from the normalized improvement in solution quality, encouraging proposals that improve the search trajectory rather than individual moves.

The model is trained using Proximal Policy Optimization (PPO) (Schulman et al. 2017). The *actor* is the two-stage policy π_θ composed of the tiny MLPs f_θ and g_θ (described above), while the *critic* is a separate small permutation-invariant MLP operating on the encoded state $\bar{s}_t$. Compared with standard PPO, our implementation incorporates an adaptive KL-regularized clipped objective, KL-driven learning-rate adaptation, and a regularized critic to improve training stability over the long stochastic SA horizon. Further details are given in Appendix A.5.

Training methodology. Beyond the PPO optimization itself, we introduce a dedicated training methodology combining diverse *initialization strategies* with *curriculum learning*.

Training instances are initialized using diverse constructive heuristics, including *random assignment*, *nearest neighbor*, *sweep*, *Clarke–Wright savings* (greedy route merging based on distance savings), and *isolate* (single-customer routes), exposing the policy to qualitatively different regions of the solution space. The construction heuristics are detailed in Appendix A.6.

During training, the reward is evaluated on short LG-SA rollouts of only a few hundred annealing steps — orders of magnitude shorter than the budgets deployed at inference. Hence, a policy trained solely from constructive solutions would rarely encounter the well-optimized states visited near the end of a long search. We therefore employ *curriculum learning* (Ma et al. 2021) to progressively shift the distribution of initial solutions encountered during training. Before each collection phase, every instance is first improved by the current policy for a limited number of LG-SA iterations, and this warm-up depth gradually increases throughout training. Consequently, early episodes focus on repairing poor initial solutions, whereas later episodes increasingly start from refined solutions and learn to perform the fine-grained improvements required near local optima. More details are given in Appendix A.7.

5 Experimental Results

5.1 Experimental Setup

Datasets. We consider three families of CVRP instances. The *design choices* of Section 5.2 are validated on synthetic instances generated following Nazari et al. (2018): customer's coordinates uniform in $[0,1]^2$, demands uniform in $\{1,\dots,9\}$, and a size-dependent capacity ($Q = 20, 30, 40, 50$ for $N = 10, 20, 50, 100$). For each size, a fixed test set of $10{,}000$ instances is pre-generated and reused by every configuration, so all comparisons are paired at the instance level.

The *baseline comparison* of Section 5.3 uses the test sets released by Ma, Cao, and Chee (2023), generated with the same procedure; since the authors re-ran a broad collection of competing methods on them under a common protocol, our numbers are directly comparable to theirs.

Finally, *generalization* is assessed on larger, real-world-structured instances: CVRPLIB Set X (Uchoa et al. 2017), the XML benchmark of Queiroga et al. (2022), and Set XL (Queiroga et al. 2026), reaching $10{,}000$ customers—two orders of magnitude beyond the training size. Unless stated otherwise, results are reported for $N = 100$.

Evaluation protocol. A trained model is evaluated by running the LG-SA loop over the full test set in parallel on a single GPU, starting from random feasible solutions, with half-precision inference and a per-experiment step budget (default 10^4). We report the mean final routing cost and the wall-clock time of the whole batch.

5.2 Design Choices

The LG-SA design space spans $K = 18$ dimensions, from the neighborhood operator and the state representation to the reward, the annealing schedule and the training curriculum, each with a finite set of candidates. The product space being far too large for exhaustive search, we explore the dimensions sequentially: at each stage, one dimension is swept while all previously adopted choices remain fixed, and its

Dimension	Adopted setting	#	Effect	App.
Operator × feasibility	**insertion + masking**	9	$+0.29^{**}$	B
Feature set	**all 13 groups**	6	$+0.34^{**}$	C
Pair descriptors	**none**	4	$+0.14^{*}$	C
Critic	**multi-statistic**	2	$+0.06^{**}$	D
Actor architecture	$\mathbf{32 \times 1}$	6	tie	D
Actor structure	**indep. scorers**	4	$+3.20^{**}$	D
Global context	**off**	2	$+0.51^{*}$	D
Distribution shaping	**logit clip C=10**	4	$+0.17^{**}$	D
Reward	**immediate**	9	tie	E
Learning rates	$\mathbf{3\times10^{-4}\,/\,10^{-3}}$	9	$+0.05^{*}$	F
Entropy coefficient	$\mathbf{0.01}$	3	$+0.13^{**}$	F
Rollout length	$\mathbf{200}$	3	tie	F
PPO passes per batch	$\mathbf{5}$	5	tie	F
Cooling shape	**geometric**	2	tie	E
Temperature range	$\boldsymbol{T_0}$**=1,** $\boldsymbol{T_{\text{final}}}$**=0.01**	6	$+0.17^{**}$	E
Metropolis	**on (train + test)**	4	$+0.12^{**}$	E
Training construction	**random**	5	$+0.22^{**}$	E
Curriculum	**ramped warm-up**	10	$+0.07^{**}$	G
End-to-end effect (N=100, 10^4 steps, cost)				
Starting point (operator winner, Table 6)			16.68 ± 0.03	
Final configuration (all winners)			TBD	
Total improvement			**TBD**	

Table 1: Summary of the design study, one row per dimension: adopted setting, number of candidates evaluated (#), and seed-paired margin of the adopted setting over the previous incumbent (*: $|\Delta| > \sigma_{\text{floor}}$; **: $|\Delta| > 2\sigma_{\text{floor}}$). Margins come from separate sweeps and are not additive; the end-to-end block reports the cumulated gain of all design choices over the first sweep's winner.

winner is carried forward. Each candidate is trained with 3 random seeds (5 for confirmations and final comparisons) and scored by the mean final cost over the test set, averaged across seeds. Comparisons are seed-paired: competing candidates share the seeds of the incumbent configuration and are compared through the per-seed cost differences Δ. A margin is significant when $|\Delta|$ exceeds a per-batch noise floor σ_{floor}, estimated from the seed-to-seed variability of stable configurations; otherwise the sweep is a tie, resolved in favor of the most stable candidate across seeds. The winning configuration is finally retrained from scratch on 5 seeds for confirmation. Table 1 summarizes the study—93 candidates across the 18 dimensions. The following paragraphs discuss the most influential design choices; Appendices B–F define every candidate and report per-candidate, per-seed results.

Confirmed choices. Four dimensions reproduce the conclusions of Andretti, Cabessa, and Strozecki (2026) and are not re-discussed here: insertion with proactive masking dominates the operator × feasibility set; Metropolis acceptance remains necessary at inference; random constructions remain the best training initialization; and widening or deepening the scoring networks never clears the noise floor, the signature of a capacity plateau (Appendices B and E, Table 13). The remaining dimensions are new to this work.

Four dimensions reproduce the conclusions of Andretti, Cabessa, and Strozecki (2026) in the present setting and are not re-discussed here. Insertion with proactive masking again dominates the operator × feasibility set, with both the lowest cost and the lowest seed variance (Appendix B). Metropolis acceptance remains necessary at inference regardless of how the actor was trained — which is also why the solver keeps sampling $a_t \sim \pi_\theta(\cdot \mid s_t)$ rather than taking the greedy argmax, a near-deterministic proposal collapsing the diversity the acceptance step relies on (Appendix E). Random constructions remain the best training initialization, while the inference-time choice is washed out by 10^4 annealing steps (Appendix E). Finally, widening or deepening the two scoring networks never clears the noise floor while wall-clock rises by up to 38% (Table 13), the signature of a capacity plateau. The remaining dimensions are new to this work.

Set	Groups	Cost	Time (s)
full set (adopted)	13	$\mathbf{16.683}$	173
no load ratio	12	16.648	173
set7	7	17.020	129
nodepct6	6	17.088	121
centroid6	6	17.100	122
core5	5	17.162	115

Table 2: Best-set validation at 5 seeds, seed-paired against the full set ($\sigma_{\text{floor}} \approx 0.057$). *core5* = static, meta, topology, neighbor ranks, detour; *set7* adds centroid and load share. Every pruned set is significantly worse; the only quality-neutral trim is dropping the load ratio alone. Paired deltas and per-seed views in Appendix C.

Node representation. The node representation is where this work departs most from Andretti, Cabessa, and Strozecki (2026). That descriptor was purely local — coordinates, depot flag, polar position, distance to the depot, and load ratios — whereas we add three groups that summarize a node's *neighborhood* rather than its own position: local density (the mean distance to the closest 5%, 10%, and 33% of customers), the insertion detour cost, and the distance to the route centroid. Every candidate feature is constrained to be computable in closed form from the coordinate and demand tensors, so the whole 13-group state is built with batched tensor operations and no per-instance loop; a feature that cannot be vectorized this way is rejected regardless of its predictive value, since it would forfeit the throughput that makes the method competitive. The additions pay for themselves, and the geometry cluster carries most of that value. Removing the detour and centroid features — both introduced here — costs $+0.596 \pm 0.033$ in the thematic ablation (Table 10), more than three times the gain of the best annealing or optimization setting and the joint-largest effect of that ablation, statistically tied with the relational cluster ($+0.599 \pm 0.016$); removing the density group costs a smaller but real $+0.145 \pm 0.033$. What the policy most needs is therefore not where a node sits, but how its neighborhood is shaped and what it would cost to move it.

The ablation also shows that individual group importance does not predict joint contribution. Leave-one-out and isolation tests (Appendix C) identify only 6 of the 13 groups

as individually useful, yet training on those alone degrades performance (Table 2); removals compound, so density and capacity statistics carry complementary information (Table 10). The only quality-neutral trim is the route load ratio (-0.035 ± 0.106); every other pruning trades quality for speed (set7 loses 2% quality for a 25% speedup). The conditional pair descriptors of the second policy stage degrade performance ($+0.14 \pm 0.11$ and $+0.18 \pm 0.07$) at extra cost. The final model therefore keeps all 13 groups and no pair descriptors.

Stabilizing the policy. Rather than enlarging the actor, we stabilize it. Logit clipping at $C = 10$ prevents the proposal distribution from collapsing and gives the larger and more consistent gain of the two mechanisms we test, at unchanged inference cost (Table 15); a multi-statistic critic improves value estimation for free, since the critic is discarded after training (Table 12). Two further extensions fail: a pooled global instance context degrades every seed at $+48\%$ compute, and a shared encoder in place of the independent scorers loses ≈ 3 routing units in every configuration (Table 14). At $N = 100$, maintaining proposal diversity and value accuracy matters more than actor capacity.

Reward. Nine shaping variants (Table 16) show that the exact form of the dense signal is secondary — the six per-step variants fall within a band of 0.07 — but that removing intermediate credit is severe: the terminal reward loses 2.95 and the step-penalty variants 0.7 to 1.1, all with unstable training. PPO needs a dense improvement signal across the long annealing horizon.

Annealing and optimization. The cooling *shape* is inert (geometric against piecewise-constant), but its *range* produces the largest gain of any setting we actually changed (the larger margins in Table 1 are losses avoided by *keeping* an existing design, not improvements): every configuration with $T_{\text{final}} = 0.01$ beats every configuration with $T_{\text{final}} = 0.1$, at every starting temperature, while the starting temperature itself does not matter (Table 17). The low-temperature phase is exactly where the learned proposal does its fine-grained refinement, and a high final temperature truncates it. Among the PPO parameters (Tables 20 and 21), the actor learning rate dominates (main effect ≈ 0.05 against a 0.031 floor), the entropy coefficient is U-shaped with an optimum at 0.01, and rollout length is asymmetric: halving it to 50 hurts clearly, doubling it to 200 helps only marginally. Retraining all selected modifications jointly adds a further -0.223 ± 0.016 across seeds (Table 23). The number of PPO passes taken over each collected batch is the one knob where quality and stability disagree (Table 22): the mean cost falls monotonically from 1 to 10 passes, but the seed spread grows with it, the 5-to-10 gain is inside that batch's 0.051 floor and sign-split, and at 15 passes training diverges outright on 2 of 4 seeds. Since the number of passes leaves inference cost unchanged, we adopt the tightest competitive setting, 5.

Curriculum. Training rewards are evaluated on LG-SA rollouts of only a few hundred steps, far shorter than the search budgets used at inference, so a policy trained only from raw constructive solutions rarely visits the fine-grained regime near local optima that dominates the end of a long search. Curriculum learning (Ma et al. 2021) closes this gap by moving the *starting point*: before each collection phase, the batch of instances is first warmed up by the current policy for t_{init} steps, and t_{init} ramps over training from 0 to a maximum warm-up strength ξ_{CL} over E epochs (Figure 4, Appendix A.7) — early training repairs poor solutions, late training refines already-improved ones. Two alternative mechanisms place the same idea elsewhere: a per-instance random depth $d_i \sim \mathcal{U}\{0, \ldots, t_{\text{init}}\}$, which makes a single batch cover the whole spectrum from raw to deeply improved solutions, and *persistent chains*, where instances and their best solutions survive across epochs and the search resumes from them (a fraction of chains being refreshed with new instances each epoch), obtaining deep starting points at no extra warm-up compute. The three mechanisms are validated against a common 5-seed no-curriculum baseline that also provides this batch's noise floor, followed by a budget-matched coverage ablation and a 5-seed confirmation of the winner.

That baseline is unusually reproducible (16.238 ± 0.006 over 5 seeds, $\sigma_{\text{floor}} = 0.030$), so a few hundredths of a routing unit is already a measurable effect. The ramped warm-up clears the floor by a factor of 2.4 (-0.073 ± 0.043, improving 4 seeds out of 5) and is the setting we adopt. Neither alternative displaces it: persistent chains are uniformly harmful across their nine runs ($+0.25$ to $+0.52$, no seed ever improving) even though they cost nothing, and the random depth is a real but smaller win over the baseline (-0.034, unanimous over 5 seeds) that loses the head-to-head against the ramp on 4 of 5 seeds. Both alternatives are monotone in the same underlying quantity — the deeper the pre-optimization, the worse the policy — and the budget-matched ablation locates the effect precisely: at equal loop length, drawing depths from $\mathcal{U}\{0, \ldots, \xi_{\text{CL}}\}$ beats warming every instance to ξ_{CL} by 0.261, but against a *constant* warm-up of the same mean depth the mix is a tie (-0.016, below the floor). What the curriculum buys is therefore a moderate warm-up rather than depth diversity per se: the actor learns from states it can still improve, and advancing the trajectory much further starves it of the reward signal it trains on (Appendix G).

5.3 Comparison with Baselines

Final model. All component winners of Table 1, the best curriculum, and the tuned optimization budget are combined into the final model, trained with 5 seeds; the best checkpoint provides the headline numbers. The comparison we control exactly is against blind SA under matched conditions — same operator, same cooling schedule, same instances, same budget — and it is reported in the "Ours" block of Table 3. Guidance is worth about two orders of magnitude of raw sampling: at $N = 100$ and $T = 10^4$, LG-SA reaches 16.18 against 22.99 for blind SA, a result the guided search already matches with 300 steps, and blind SA still sits at 18.13 with ten times the moves — which LG-SA passes at 10^3. Comparing complete, independently annealed runs at every budget from 10^2 to 10^5 steps (Appendix H, Figure 25), the guided search is ahead at every point, by 27.7 at 10^2 down to 2.2 at 10^5. The advantage is not free per move — the actor

forward pass makes a guided step 13.1 to 13.7× more expensive than a blind one — but it survives the conversion to wall clock: LG-SA at 10^4 steps (16.18 in 2.9m) against blind SA at 10^5 (18.13 in 2.3m) is a 1.95 margin at comparable cost, and blind SA's best measured point is beaten by a guided run taking 7.9× less time. That margin grows with instance size, from 0.19 at $N = 20$ to 1.69 at $N = 50$ and 6.81 at $N = 100$ under an equal step budget: the larger the neighborhood, the more there is to gain from choosing where to look. Test-time augmentation offers a second way to spend compute: running the search over the eight dihedral symmetries of the instance ($A = 8$) and keeping the best frame lowers the $T = 10^4$ gap to 0.47%, 1.16% and 2.36% at $N = 20$, 50 and 100 (Table 3), matching the ten-times-longer single run at $T = 10^5$ on $N = 100$ and beating it on the two smaller sizes, at lower wall-clock cost — so augmentation and raw budget buy comparable quality, and the choice between them is one of latency against throughput.

Literature baselines. Table 3 places LG-SA in the context of the published results on the Nazari benchmark. We group the baselines into four families. *Conventional solvers* — HGS (Vidal 2022) and LKH-3 (Helsgaun 2017) — delimit the quality/time frontier reachable without learning. HGS, a hybrid genetic search combining a diversity-managed population with an aggressive local search over the SWAP* neighborhood, is the stronger of the two and is the reference metaheuristic for the CVRP; we therefore report all gaps with respect to it, following Ma, Cao, and Chee (2023). The general-purpose solver OR-Tools (Furnon and Perron 2024), a common reference in earlier comparisons, is not re-run by that source and is omitted here; it is dominated by both HGS and LKH-3 on this distribution. *Construction methods* — AM (Kool, van Hoof, and Welling 2019) with collaborative policies (Kim, Park, and Kim 2021), POMO (Kwon et al. 2021), and its search-augmented variants EAS (Hottung, Kwon, and Tierney 2021) and SGBS (Choo et al. 2022) — build a solution autoregressively and obtain most of their quality from test-time sampling, beam search, or instance augmentation. *Improvement methods* — NLNS (Hottung and Tierney 2022), NCE (Kim, Park, and Kim 2021), LIH (Wu et al. 2022), DACT (Ma et al. 2021), and NeuOpt (Ma, Cao, and Chee 2023) — iteratively refine a solution and are the family LG-SA belongs to; for these we report the step budget T, which controls the quality/time trade-off. Finally, *hybrid methods* — DPDP (Kool et al. 2022) and CVAE-Opt (Hottung, Bhandari, and Tierney 2021) — couple a learned component with dynamic programming or an evolutionary search.

Reading the comparison. We source the baseline values from Ma, Cao, and Chee (2023) rather than from the older aggregation of Ma et al. (2021), because the former re-runs every baseline from its public pre-trained model on a single common test set and common hardware, instead of collecting heterogeneous numbers from the original papers. This makes the objective and time columns internally consistent, and it supplies HGS at all three sizes. Two caveats remain. First, rows marked ‡ could not be re-run and are carried over from their original papers; their values are approximate and their run times were measured elsewhere. Second, run times still mix one GPU for the neural methods with one CPU for HGS and LKH-3, and they are totals over the whole 10,000-instance set, so the time column separates orders of magnitude rather than factors of two. The prior LG-SA rows (Andretti, Cabessa, and Strozecki 2026) carry the additional caveat of a different instance draw, marked †. Against this backdrop, the comparison we control exactly is LG-SA against blind SA under matched conditions — same operator, step budget, cooling schedule, and instances — which is reported in the same block.

5.4 Generalization and Scalability

Every number reported so far comes from one checkpoint, trained at $N = 100$ with capacity $Q = 50$ and never retrained per size. The $N = 20$ and $N = 50$ columns of Table 3 are therefore already zero-shot transfers, run at capacities ($Q = 30$ and $Q = 40$) the policy never saw during training, and the gap there is smaller than at the training size (0.85% and 1.55% against 2.31% at $T = 10^5$). Smaller instances are easier for every method, so that ordering is not by itself evidence of good transfer; what it does establish is that no size-specific retraining is needed to stay competitive away from the training size.

Upward, the same checkpoint is run sequentially on single instances at sizes up to $N = 10{,}000$, two orders of magnitude above training (Table 4). Nothing breaks: the search stays feasible and improves the constructive solution by 64–75% at every size, and the per-step cost grows by only 7.2× while N grows by 100× — sublinear, because below $N \approx 10^3$ a step is dominated by fixed overhead rather than by the instance. A step budget fixed in absolute terms therefore gets cheaper per node but also thinner: at $N = 10{,}000$, 10^4 steps is one proposed move per node, which is where the improvement ratio starts to fall back. Halving the exponent makes this explicit — at 10^3 steps the improvement collapses from −68% at $N = 100$ to −14% at $N = 10{,}000$, and is recovered by spending 10× more steps (Appendix H). The currency is steps per node, not steps; what degrades at scale is the budget, not the representation. Appendix H also reports the step-budget, batch-size and precision sweeps behind the evaluation protocol used throughout the paper — in particular that half precision is free (1.29× faster for −0.0002 in cost), which is why every reported evaluation uses it.

6 Conclusion

Family	Method	$N = 20$			$N = 50$			$N = 100$		
		Obj.	**Gap**	**Time**	**Obj.**	**Gap**	**Time**	**Obj.**	**Gap**	**Time**
Conventional	HGS (reference)	6.130	—	10.7h	10.366	—	1.2d	15.563	—	2.5d
	LKH-3	6.135	0.08%	17.9h	10.375	0.09%	2.8d	15.647	0.54%	5.7d
Construction	AM+LCP‡	6.15	0.33%	23m	10.52	1.48%	52m	16.00	2.81%	2.1h
	POMO (A=8)	6.136	0.09%	11m	10.397	0.30%	1.4h	15.672	0.70%	7.2h
	POMO+EAS	6.132	0.04%	38m	10.379	0.13%	3.1h	15.610	0.30%	16h
	POMO+EAS+SGBS	—	—	—	—	—	—	15.579	0.10%	4.1d
Improvement	NLNS (T=5k)	6.175	0.73%	48m	10.506	1.35%	1.4h	15.915	2.26%	2.4h
	NCE‡	6.13	0.00%	11h	10.41	0.42%	2.3d	15.81	1.59%	10.4d
	LIH (T=5k)	—	—	—	10.544	1.72%	4.2h	16.165	3.87%	5h
	DACT (A=6, T=10k)	6.130	0.01%	4h	10.383	0.16%	16h	15.736	1.11%	1.7d
	NeuOpt (D=1, T=10k)	6.130	0.00%	41m	10.375	0.08%	2h	15.656	0.60%	4.6h
	NeuOpt (D=5, T=40k)	6.130	0.00%	13.7h	10.367	0.01%	1.6d	15.579	0.10%	3.8d
Hybrid	CVAE-Opt-DE‡	6.14	—	2.4d	10.40	—	4.7d	15.75	—	11d
	DPDP (1000k)	—	—	—	—	—	—	15.627	0.41%	1.2d
Prior LG-SA	LG-SA (T=10^4)	—	—	—	11.02	6.31%†	87s	17.36	11.55%†	3m
	LG-SA (T=10^5)	—	—	—	10.70	3.22%†	14m	16.80	7.95%†	29m
	LG-SA (T=10^6)	—	—	—	10.54	1.68%†	2.3h	—	—	—
Ours	Blind SA (T=10^4)	6.438	5.02%	7s	12.351	19.15%	7s	22.992	47.73%	14s
	Blind SA (T=10^5)	6.237	1.75%	1.1m	11.122	7.29%	1.2m	18.132	16.51%	2.3m
	LG-SA (T=10^4)	6.246	1.89%	21s	10.666	2.89%	1.5m	16.183	3.98%	3.0m
	LG-SA (T=10^4, C–W)	6.213	1.36%	24s	10.617	2.42%	1.5m	16.117	3.56%	3.2m
	LG-SA (T=10^4, A=8)	6.159	0.47%	2.8m	10.487	1.16%	11.5m	15.930	2.36%	23m
	LG-SA (T=10^5)	6.182	0.85%	3.5m	10.526	1.55%	14.6m	15.923	2.31%	29.9m
	LG-SA (T=10^6)	6.150	0.32%	34m	10.452	0.83%	2.3h	TBD	TBD	TBD

Table 3: Comparison with published baselines on the Nazari CVRP benchmark. Unless marked otherwise, values are taken from Ma, Cao, and Chee (2023), who re-run every baseline from its public pre-trained model on a common test set of 10,000 instances and common hardware (one GPU for neural methods, one CPU for conventional ones); gaps are relative to HGS, the strongest conventional solver. ‡Carried over from the original paper rather than re-run, so their objectives are rounded to two decimals and their run times were measured on different hardware. †Prior LG-SA values are from Andretti, Cabessa, and Strozecki (2026), measured on a different draw of the same distribution; their gaps are recomputed against the HGS values above and are therefore indicative only. A and D denote the number of instance augmentations, T the number of improvement steps; all LG-SA rows start from a random construction except "C–W", which starts from a Clarke–Wright savings solution (Appendix E).

N	Init.	LG-SA	Improv.	ms/step
100	59.4	18.3	−69%	0.46
200	119.2	31.3	−74%	0.51
500	288.8	75.3	−74%	0.62
1,000	588.5	147.9	−75%	0.79
2,000	1188.4	309.4	−74%	1.09
5,000	2944.6	850.3	−71%	1.91
10,000	5866.4	2100.6	−64%	3.32

Table 4: Scaling beyond the training size. The $N = 100$ checkpoint is applied unchanged to instances up to $N = 10{,}000$, sampled from the Nazari distribution with the capacity held at the training value $Q = 50$ (so larger N means proportionally more routes). Single instances, batch 1, CPU, 10^4 steps, mean over 3 runs; instance construction is timed separately and excluded. Costs at different N are not comparable to each other, only to their own constructive initialization.

References

Accorsi, L.; and Vigo, D. 2021. A Fast and Scalable Heuristic for the Solution of Large-Scale Capacitated Vehicle Routing Problems. *Transportation Science*, 55(4): 832–856.

Andretti, J.; Cabessa, J.; and Strozecki, Y. 2026. Learning-Guided Simulated Annealing for the Capacitated Vehicle Routing Problem. In *Proceedings of the International Conference on Automated Planning and Scheduling*, volume 36. AAAI Press.

Bui, V.; and Mai, T. 2023. Imitation Improvement Learning for Large-Scale Capacitated Vehicle Routing Problems. *Proceedings of the International Conference on Automated Planning and Scheduling*, 33: 551–559.

Chen, X.; and Tian, Y. 2019. Learning to Perform Local Rewriting for Combinatorial Optimization. In *Advances in Neural Information Processing Systems*, volume 32. Curran Associates, Inc.

Choo, J.; Kwon, Y.-D.; Kim, J.; Jae, J.; Hottung, A.; Tierney, K.; and Gwon, Y. 2022. Simulation-guided Beam Search for Neural Combinatorial Optimization. In *Advances in Neural Information Processing Systems*, volume 35, 8760–8772. Curran Associates, Inc.

Clarke, G.; and Wright, J. W. 1964. Scheduling of Vehicles from a Central Depot to a Number of Delivery Points. *Operations Research*, 12(4): 568–581.

Correia, A. H. C.; Worrall, D. E.; and Bondesan, R. 2023. Neural Simulated Annealing. In *Proceedings of The 26th International Conference on Artificial Intelligence and Statistics*, 4946–4962. PMLR. ISSN: 2640-3498.

Dantzig, G. B.; and Ramser, J. H. 1959. The Truck Dispatching Problem. *Management Science*, 6(1): 80–91.

Drakulic, D.; Michel, S.; Mai, F.; Sors, A.; and Andreoli, J.-M. 2023. BQ-NCO: Bisimulation Quotienting for Efficient Neural Combinatorial Optimization. In *Advances in Neural Information Processing Systems*, volume 36, 77416–77429. Curran Associates, Inc.

Furnon, V.; and Perron, L. 2024. OR-Tools Routing Library.

García-Torres, R.; Macias-Infante, A. A.; Conant-Pablos, S. E.; Ortiz-Bayliss, J. C.; and Terashima-Marín, H. 2022. Combining Constructive and Perturbative Deep Learning Algorithms for the Capacitated Vehicle Routing Problem. ArXiv:2211.13922 [cs].

Helsgaun, K. 2017. An Extension of the Lin-Kernighan-Helsgaun TSP Solver for Constrained Traveling Salesman and Vehicle Routing Problems. Publisher: 398.

Hottung, A.; Bhandari, B.; and Tierney, K. 2021. Learning a Latent Search Space for Routing Problems using Variational Autoencoders. In *International Conference on Learning Representations*.

Hottung, A.; Kwon, Y.-D.; and Tierney, K. 2021. Efficient Active Search for Combinatorial Optimization Problems.

Hottung, A.; and Tierney, K. 2022. Neural large neighborhood search for routing problems. *Artificial Intelligence*, 313: 103786.

Kim, M.; Park, J.; and Kim, J. 2021. Learning Collaborative Policies to Solve NP-hard Routing Problems. In *Advances in Neural Information Processing Systems*, volume 34, 10418–10430. Curran Associates, Inc.

Kirkpatrick, S.; Gelatt, C. D.; and Vecchi, M. P. 1983. Optimization by Simulated Annealing. 220.

Kool, W.; van Hoof, H.; Gromicho, J.; and Welling, M. 2022. Deep Policy Dynamic Programming for Vehicle Routing Problems. In *Integration of Constraint Programming, Artificial Intelligence, and Operations Research (CPAIOR)*, 190–213. Springer.

Kool, W.; van Hoof, H.; and Welling, M. 2019. Attention, Learn to Solve Routing Problems! In *International Conference on Learning Representations*.

Kwon, Y.-D.; Choo, J.; Kim, B.; Yoon, I.; Gwon, Y.; and Min, S. 2021. POMO: Policy Optimization with Multiple Optima for Reinforcement Learning. ArXiv:2010.16011 [cs].

Liu, F.; Lu, C.; Gui, L.; Zhang, Q.; Tong, X.; and Yuan, M. 2023. Heuristics for Vehicle Routing Problem: A Survey and Recent Advances. Version Number: 1.

Lu, H.; Zhang, X.; and Yang, S. 2020. A Learning-based Iterative Method for Solving Vehicle Routing Problems. In *International Conference on Learning Representations*.

Luo, F.; Lin, X.; Liu, F.; Zhang, Q.; and Wang, Z. 2023. Neural Combinatorial Optimization with Heavy Decoder: Toward Large Scale Generalization. In *Advances in Neural Information Processing Systems*, volume 36, 8845–8864. Curran Associates, Inc.

Ma, Y.; Cao, Z.; and Chee, Y. M. 2023. Learning to Search Feasible and Infeasible Regions of Routing Problems with Flexible Neural k-Opt. ArXiv:2310.18264 [cs].

Ma, Y.; Li, J.; Cao, Z.; Song, W.; Zhang, L.; Chen, Z.; and Tang, J. 2021. Learning to Iteratively Solve Routing Problems with Dual-Aspect Collaborative Transformer. In *Advances in Neural Information Processing Systems*, volume 34, 11096–11107. Curran Associates, Inc.

Nazari, M.; Oroojlooy, A.; Snyder, L.; and Takac, M. 2018. Reinforcement Learning for Solving the Vehicle Routing Problem. In *Advances in Neural Information Processing Systems*, volume 31. Curran Associates, Inc.

Queiroga, E.; Martinelli, R.; Subramanian, A.; Uchoa, E.; and Vidal, T. 2026. The XL Instances for the Capacitated Vehicle Routing Problem. ArXiv:2601.11467 [math].

Queiroga, E.; Sadykov, R.; Uchoa, E.; and Vidal, T. 2022. 10,000 optimal CVRP solutions for testing machine learning based heuristics. In *AAAI-22 Workshop on Machine Learning for Operations Research (ML4OR)*.

Schulman, J.; Wolski, F.; Dhariwal, P.; Radford, A.; and Klimov, O. 2017. Proximal Policy Optimization Algorithms. ArXiv:1707.06347 [cs].

Toth, P.; and Vigo, D., eds. 2002. *The vehicle routing problem*. SIAM monographs on discrete mathematics and applications. Philadelphia, Pa: Society for Industrial and Applied Mathematics. ISBN 978-0-89871-498-2 978-0-89871-579-8.

Uchoa, E.; Pecin, D.; Pessoa, A.; Poggi, M.; Vidal, T.; and Subramanian, A. 2017. New benchmark instances for the Capacitated Vehicle Routing Problem. *European Journal of Operational Research*, 257(3): 845–858.

Vidal, T. 2016. Technical note: Split algorithm in O(n) for the capacitated vehicle routing problem. *Computers & Operations Research*, 69: 40–47.

Vidal, T. 2022. Hybrid genetic search for the CVRP: Open-source implementation and SWAP* neighborhood. *Computers & Operations Research*, 140: 105643.

Vinyals, O.; Fortunato, M.; and Jaitly, N. 2015. Pointer Networks. In *Advances in Neural Information Processing Systems*, volume 28. Curran Associates, Inc.

Wu, X.; Wang, D.; Wen, L.; Xiao, Y.; Wu, C.; Wu, Y.; Yu, C.; Maskell, D. L.; and Zhou, Y. 2024. Neural Combinatorial Optimization Algorithms for Solving Vehicle Routing Problems: A Comprehensive Survey with Perspectives. ArXiv:2406.00415.

Wu, Y.; Song, W.; Cao, Z.; Zhang, J.; and Lim, A. 2022. Learning Improvement Heuristics for Solving Routing Problems. *IEEE Trans. Neural Networks Learn. Syst.*, 33(9): 5057–5069.

Zhu, R.; Zhang, C.; and Cao, Z. 2025. Refining Hybrid Genetic Search for CVRP via Reinforcement Learning-Finetuned LLM.

Appendix Overview

The appendices mirror the organization of the main text. Appendix A expands the Model section component by component: the node feature definitions, the two-stage neural policy, the neighborhood operators, the feasibility strategies, the reinforcement-learning formulation with our PPO modifications, the constructive initialization heuristics, and the curriculum mechanisms. Appendices B–G then carry the evidence behind the Design Choices subsection, one appendix per design dimension in sweep order: operators and feasibility (B), node features (C), architecture (D), the search-loop components — reward, annealing, acceptance, initialization — (E), the optimization hyperparameters, including the per-seed record and mechanism of the combined-configuration divergence (F), and the curriculum study (G). Each full-results appendix collects the per-dimension motivation and candidate definitions, the complete configuration comparisons and secondary tables with paired deltas and per-batch noise floors, and the per-seed diagnostic plots behind the master summary (Table 1) and the two headline tables, and opens with the mapping from the internal configuration names appearing in the plot legends to the paper terminology. Two conventions are shared throughout: every comparison is seed-paired (both arms are trained on the same seeds), and every delta plot shows the per-seed differences against a shaded band marking that batch's noise floor — an effect is trusted only if it clears the band with a consistent sign across seeds.

A Model Details

This appendix expands the Model section of the main text, component by component and in the same order: the node feature representation (A.1), the two-stage neural policy (A.2), the neighborhood operators (A.3), the feasibility strategies (A.4), the reinforcement-learning formulation and our PPO modifications (A.5), the constructive initialization heuristics (A.6), and the curriculum mechanisms (A.7).

A.1 Node Features

This section defines every component of the node feature vector $\mathbf{x}_i$ used by the policy and the critic. For a given solution s_t, $\mathrm{prev}(i)$ and $\mathrm{succ}(i)$ denote the nodes immediately before and after i in the current visiting order, $r(i)$ the route currently serving i, $Q_{r(i)}$ the total load of that route, and $d(\cdot,\cdot)$ the Euclidean distance. Figure 1 illustrates the geometric quantities, and Table 5 summarizes the groups and their dimensions. All features are normalized to comparable ranges, and each is computed locally at node i: the encoding contains no fixed-size global descriptor, which is what makes the representation applicable to any instance size.

Static instance features (6). The 2D coordinates $\mathbf{p}_i = (p_i^x, p_i^y) \in [0,1]^2$; a depot indicator $\mathbf{1}_{\{i=0\}}$; the polar angle θ_i of the node around the depot; the distance to the depot d_{i0}, min–max normalized per instance; and the demand normalized by the vehicle capacity, q_i/Q. These features depend only on the instance, not on the current solution.

Route topology (4). The coordinates $\mathbf{p}_{\mathrm{prev}(i)}$ and $\mathbf{p}_{\mathrm{succ}(i)}$ of the node's current neighbors in the visiting order. They give the policy direct access to the local shape of the route around each node.

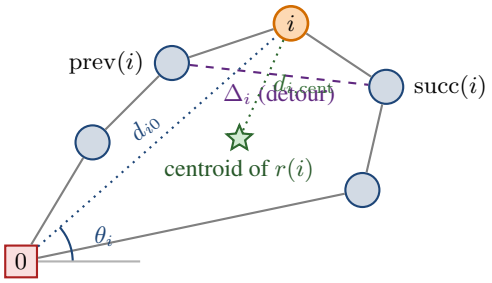

Figure 1: Geometric features attached to a node i (orange) within its current route. The detour Δ_i measures the extra length caused by visiting i between its neighbors; (θ_i, d_{i0}) locate i in polar coordinates around the depot; $d_{i,\mathrm{cent}}$ measures how far i lies from the center of gravity of its own route.

Neighbor distance-ranks (2). For each of $\mathrm{prev}(i)$ and $\mathrm{succ}(i)$, the rank of that neighbor in the list of all nodes sorted by distance from i, normalized to $[0,1]$ (the nearest node has rank ≈ 0). A well-placed node is linked to low-rank neighbors; a high rank signals a locally poor connection, independently of the absolute scale of the instance.

Relative demand (1). The demand normalized by the largest demand in the instance, $q_i / \max_{j \in \mathcal{C}} q_j$, complementing q_i/Q with a signal that is invariant to the capacity level.

Spatial density (3). The mean distance from i to its k nearest neighbors, evaluated at three scales: $k = 5$, $k = \lceil N/10 \rceil$, and $k = \lceil N/3 \rceil$. These static statistics distinguish nodes in dense clusters from isolated ones.

Local solution quality (2). The *detour cost*

$$\Delta_i = d(\mathrm{prev}(i), i) + d(i, \mathrm{succ}(i)) - d(\mathrm{prev}(i), \mathrm{succ}(i)),$$

i.e., the extra length the tour pays for visiting i at its current position (equivalently, the saving obtained by removing it); and the distance $d_{i,\mathrm{cent}}$ from i to the centroid of the nodes of its route, which flags spatial outliers assigned to the wrong route.

Capacity status (3). The load ratio of the node's route, $L_{r(i)} = Q_{r(i)}/Q$; the remaining slack, $S_{r(i)} = (Q - Q_{r(i)})/Q$; and the node's share of its route load, $\eta_i = q_i/Q_{r(i)}$. These dynamic signals indicate whether a route is nearly saturated and how much a given node contributes to that saturation.

Search meta-features (2). The current SA temperature, normalized to $[0,1]$ over the schedule range, and the fraction of the search budget remaining. They are identical across nodes of the same state and let the policy modulate its behavior along the annealing schedule (e.g., bolder restructuring at high temperature, fine repairs near the end).

Conditional pair features (stage 2, optional). In addition to the per-node vectors above, the second stage of the policy

Group	Features	Dim
Static	$\mathbf{p}_i$, $\mathbf{1}_{\{i=0\}}$, θ_i, d_{i0}, q_i/Q	6
Topology	$\mathbf{p}_{\mathrm{prev}(i)}$, $\mathbf{p}_{\mathrm{succ}(i)}$	4
Neighbor ranks	rank of $\mathrm{prev}(i)$, $\mathrm{succ}(i)$	2
Relative demand	$q_i / \max_j q_j$	1
Spatial density	μ_5, $\mu_{N/10}$, $\mu_{N/3}$	3
Local quality	Δ_i, $d_{i,\mathrm{cent}}$	2
Capacity status	$L_{r(i)}$, $S_{r(i)}$, η_i	3
Meta	temperature, remaining budget	2
Total		23

Table 5: Summary of the node feature vector $\mathbf{x}_i$.

can receive descriptors of the candidate move itself, computed only for the selected first node u and every candidate partner v: the exact insertion cost $d(v,u) + d(u,\mathrm{succ}(v)) - d(v,\mathrm{succ}(v))$ together with the net cost change of the full move (insertion cost minus the removal saving of u), and the normalized distance-ranks of the edges the move would create. Because they are evaluated for a single u, these descriptors preserve the $\mathcal{O}(N)$ per-step complexity. These optional descriptors are evaluated experimentally in Appendix C and are not part of the final model.

A.2 Two-Stage Neural Policy

The two-stage neural policy π_θ, parametrized by the neural networks f_θ and g_θ, is illustrated in Figure 2. Both networks are small multilayer perceptrons applied to every node (stage 1) or every candidate pair (stage 2) in parallel, as a single batched tensor operation: a linear input layer projecting to the embedding width, a stack of hidden layers of that width with LeakyReLU activations, and a bias-free linear output layer producing one scalar score per node. The adopted configuration uses an embedding width of 32 with a single hidden layer; the architecture study of Appendix D shows that larger networks bring no significant gain. The input of f_θ is the node feature vector $\mathbf{x}_k$ of Appendix A.1; the input of g_θ concatenates the feature vector of the selected node i with that of each candidate partner, dropping the duplicated search meta-features, which are identical for all nodes of a given state. The output layers are initialized orthogonally with a small gain, so the initial proposal distribution is near-uniform and early training explores like blind SA rather than following an arbitrary initial policy. Following Kool, van Hoof, and Welling (2019), the scores of both stages are bounded by $C\tanh(\cdot)$ with $C = 10$ before the softmax, which prevents the proposal distribution from collapsing to a near-deterministic choice (validated in Appendix D).

A.3 Neighborhood Operators

We consider the following standard neighborhood operators:

- *Swap:* exchanges the positions of nodes i and j, either within the same route or across two routes;
- *Insertion:* relocates node i immediately after node j;
- *2-opt / 2-opt*:* for intra-route pairs, reverses the segment between i and j; for inter-route pairs, exchanges the route suffixes following i and j.

Figure 3 illustrates the three pairwise operators on two route fragments. The same action $a = (i,j)$ triggers a qualitatively different transformation under each operator, which is why a policy is trained for one fixed operator and learns a proposal distribution tailored to it.

A.4 Feasibility Strategies

Under capacity constraints, many actions produce an infeasible neighbor $H(s,a) \notin \mathcal{F}$ — the central obstacle in extending neural SA from unconstrained problems to the CVRP. We compare three ways of keeping the search inside $\mathcal{F}$:

- *Proactive filtering (action masking).* The policy is restricted to the feasible subset $\mathcal{A}_f(s) = \{a : H(s,a) \in \mathcal{F}\}$ by masking infeasible actions before sampling. The conditional factorization makes this cheap: the first node i is selected freely, and only then are the feasible partners $\{j : H(s,(i,j)) \in \mathcal{F}\}$ computed and used to mask the second stage — $\mathcal{O}(N)$ checks per step instead of $\mathcal{O}(N^2)$. Every proposal is feasible by construction; since the fleet is unlimited, a node can always open a new route, so the feasible set is never empty.
- *Reactive rejection.* The policy samples any action from its learned distribution and the resulting neighbor is checked afterwards: if $s' \notin \mathcal{F}$, the move is rejected and the current solution retained. This avoids the filtering overhead but can waste both compute and learning signal when many sampled moves are infeasible.
- *Indirect representation.* Feasibility is delegated to the representation: the solution is encoded as an unconstrained permutation of the customers, the policy modifies the permutation, and a deterministic *greedy split* heuristic packs it back into capacity-feasible routes (Vidal 2016). Every state is feasible by construction and the policy never handles constraints, but the reconstruction runs at every step and its cost grows with the problem size.

A.5 Reinforcement Learning and PPO

The model is trained with Reinforcement Learning (RL) using *Proximal Policy Optimization* (PPO), the standard on-policy actor-critic algorithm (Schulman et al. 2017). To this end, we frame the LG-SA loop as a Markov Decision Process (MDP). At step t, the *state* is the encoded representation of the feasible solution $\bar{s}_t$. The *action* is the node pair $a_t = (i,j)$ handed to the operator H. The *transition* is the composition of the operator and the stochastic Metropolis acceptance: the next state is $H(s_t, a_t)$ if the move is accepted and s_t otherwise, so the environment dynamics include the acceptance randomness of SA.

The *reward* is derived from the evolution of the solution cost along the search: a move that is accepted and improves the solution receives a positive reward proportional to the (normalized) cost reduction, a rejected move yields no improvement signal, and proposals leading to infeasible neighbors can be penalized. Reward variants that instead emphasize improvements of the best solution found so far, or the final solution quality at the end of the horizon, fit the same framework; the precise shaping is a design choice studied experimentally. The agent thus maximizes the expected

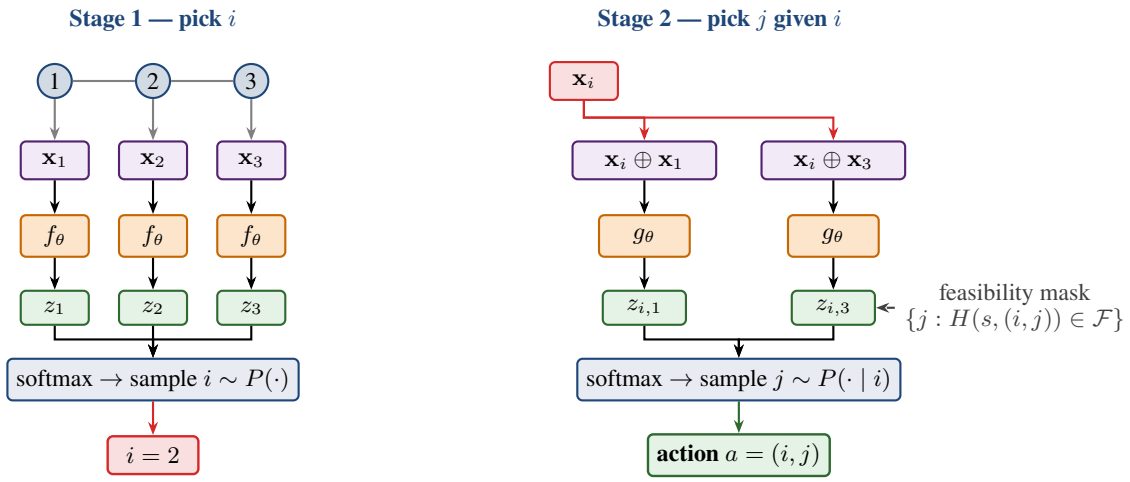

Figure 2: Two-stage conditional sampling of the node pair (i, j), shown on a toy route of three customers. Stage 1: every node's feature vector is scored independently by a shared network f_θ and the first node i is sampled from the softmax over the scores. Stage 2: the representation of the selected node i is combined with that of every candidate partner j and scored by a second network g_θ; infeasible partners are masked out before the softmax, and j is sampled from the conditional distribution. The factorization $P(i, j) = P(i) \cdot P(j \mid i)$ makes each proposal $\mathcal{O}(N)$.

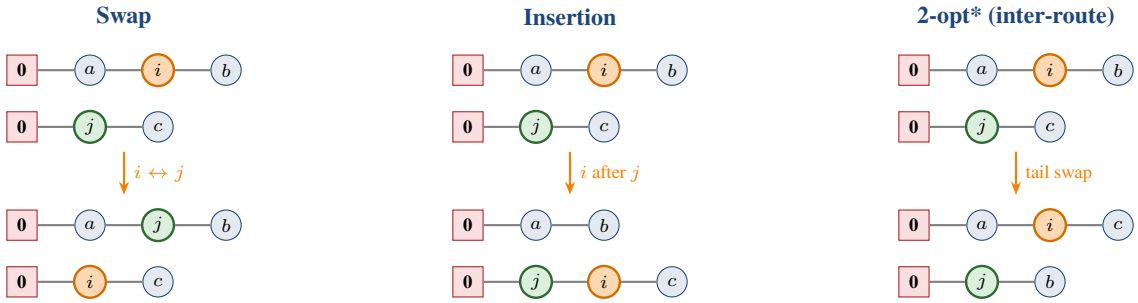

Figure 3: The three pairwise operators on two route fragments (top: before, bottom: after). *Swap* exchanges i (orange) and j (green). *Insertion* removes i and reinserts it immediately after j. *2-opt** exchanges the route suffixes following i and j; when i and j belong to the same route, the operator instead reverses the path segment between them (standard 2-opt).

cumulative discounted reward over the SA horizon, which aligns the learned proposal with the overall goal of driving the annealing process toward low-cost solutions rather than with any single greedy step.

The policy is trained with PPO. Training alternates between a *collection* phase, in which the current policy runs the full LG-SA loop of Algorithm 1 on a large batch of freshly generated instances in parallel and stores every transition (s_t, a_t, r_t), and an *update* phase, in which the stored episodes are replayed for several optimization epochs. Advantages are estimated with Generalized Advantage Estimation, and the actor maximizes the usual clipped surrogate objective with an entropy bonus; we refer to Schulman et al. (2017) for the standard formulation and only detail our departures from it.

The critic V_ϕ is a separate permutation-invariant network: it encodes each node feature vector independently, pools the embeddings across nodes into a fixed-size summary, and maps this summary to a scalar value estimate. Keeping the actor and critic distinct prevents interference between the policy and value representations.

Our implementation departs from standard PPO in three ways, all aimed at stabilizing training on the long and noisy SA horizon:

- *Hybrid clipping–KL objective.* Rather than choosing between the clipped surrogate and the KL-penalized objective, we use both: an adaptive Kullback–Leibler penalty $\beta_{\mathrm{KL}}\, D_{\mathrm{KL}}(\pi_{\theta_{\mathrm{old}}} \| \pi_\theta)$ is added to the clipped loss, and its coefficient is doubled or halved after each update epoch depending on whether the measured divergence leaves a band around a target value. This soft trust region allows many optimization epochs on each collected batch without catastrophic policy drift.
- *KL-coupled learning rate.* The same divergence signal modulates the actor's learning rate within fixed bounds — gently decreased when the policy moves too fast, increased when it stalls — so the effective step size tracks the trust region automatically.
- *Regularized critic.* The value loss is clipped, mirroring

the actor's trust region, and augmented with a gradient penalty that encourages a smooth (near-Lipschitz) value landscape; we found this useful given that consecutive states differ by a single local move whose acceptance is itself stochastic.

A.6 Initialization Strategies

Every training episode and every evaluation starts from a feasible solution produced by one of five constructive heuristics; the parenthesized numbers give their mean construction cost on the $N = 100$ test set, spanning a sixfold range:

- *Random assignment* (57.9): the customers are placed in a uniformly random order and the sequence is split greedily into routes, a depot return being inserted whenever the next customer would exceed the remaining capacity.
- *Nearest neighbor* (19.0): routes are built one at a time by repeatedly visiting the closest unvisited customer, returning to the depot when no remaining customer fits the residual capacity.
- *Sweep* (30.5): the customers are sorted by polar angle around the depot and inserted in angular order, a new route being opened whenever the capacity would be exceeded.
- *Clarke–Wright savings* (16.4): starting from one dedicated route per customer, pairs of routes are iteratively merged in decreasing order of the distance saving achieved by the merge, subject to capacity feasibility.
- *Single-customer routes* (104.0): each customer is served by its own dedicated route — the simplest feasible construction, and by far the most expensive one.

The training-time construction fixes the state distribution the policy learns from, while the test-time construction is a free knob of the solver; the two roles are studied separately in Appendix E.

A.7 Curriculum Learning

Training episodes are much shorter than the search budgets used at inference, so a policy trained only from raw constructive solutions rarely visits the fine-grained regime near local optima that dominates the end of a long search. The curriculum (Ma et al. 2021) closes this gap by moving the *starting point*: before each collection phase, the batch of instances is first warmed up by the current policy for t_{init} steps, and t_{init} grows over training from 0 to a maximum warm-up strength ξ_{CL} over E epochs, following a linear or sigmoid ramp (Figure 4). Early in training the policy thus learns to repair poor solutions; later it increasingly learns to improve already-refined ones. A second mechanism draws a per-instance depth $d_i \sim \mathcal{U}\{0, \ldots, t_{\text{init}}\}$ instead of warming every instance to the same depth, so that a single batch covers the entire spectrum from raw initializations to deeply improved solutions; it is implemented by freezing an instance once it passes d_i, so an arm of nominal strength ξ_{CL} costs the same wall-clock whether its depth is drawn or constant, and only the *effective* mean depth $\xi_{\text{CL}}/2$ differs. A third mechanism, *persistent chains*, removes the warm-up phase altogether: instances and their incumbent best solutions survive from one epoch to the next and the search resumes

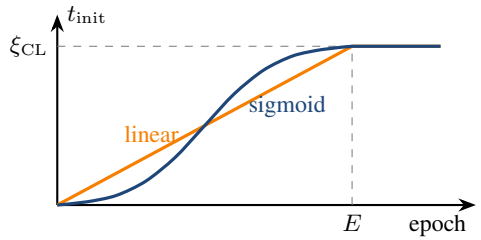

Figure 4: Curriculum ramp. Before each experience-collection phase, the batch of instances is pre-improved by running the current policy for t_{init} SA steps, and training then starts from those warmed-up solutions instead of raw constructive ones. The plot shows how that warm-up depth is scheduled: t_{init} grows from 0 (train on raw solutions) to the warm-up strength ξ_{CL} (train on deeply improved ones) over the first E epochs, either linearly or along a sigmoid, and stays at ξ_{CL} afterwards. The x-axis is training epochs, not SA steps.

where it stopped, a fraction ρ of the chains being replaced by fresh instances each epoch, so the mean resume depth is $\text{OUTER\_STEPS}/\rho$ SA steps and the mechanism is free.

The curriculum is validated with its own sequential protocol: (0) a 5-seed no-curriculum baseline, which also provides the noise floor used throughout the curriculum study; (1) the ramped warm-up, at the strength, ramp duration and steepness selected in preliminary work, re-validated against that baseline; (2) a sweep of the warm-up strength under the stochastic depth, with the ramp disabled; (3) a sweep of the chain refresh rate ρ; (4) a budget-matched coverage ablation opposing the stochastic depth to a constant warm-up of the same loop length and of the same mean depth; and (5) a 5-seed head-to-head confirmation between the two surviving mechanisms. Full results are given in Appendix G.

B Operator–Feasibility Study, Full Results

This appendix collects the complete results of the operator $\times$ feasibility sweep (Table 6); the operators and feasibility strategies themselves are defined in Appendices A.3 and A.4. The sweep replicates, under the present training configuration, the operator–feasibility study of Andretti, Cabessa, and Strozecki (2026), and reaches the same conclusion. It trains one policy per operator–strategy configuration (3 seeds per configuration, 27 runs). In the plot legends, the internal configuration names map to the paper terminology as `valid` = proactive masking, `free` = indirect representation (greedy split), `rm_depot` = reactive rejection.

Main effects. Table 7 averages each factor level over the other factor. Insertion and masking win marginally as well as jointly, so the winning configuration of Table 6 is not an interaction artefact: the best operator and the best feasibility strategy are individually best as well. Relocating a single node is the move that best matches a learned pointer to a promising node pair, whereas the value of a swap or a segment reversal depends more delicately on both endpoints simultaneously, making the credit assignment noisier

Operator	Masking	Indirect	Rejection
Insertion	$\mathbf{16.68 \pm 0.03}$	16.97 ± 0.04	26.36 ± 0.71
Swap	19.50 ± 1.54	22.82 ± 1.36	35.75 ± 3.04
2-opt(*)	18.93 ± 0.09	21.29 ± 0.39	25.84 ± 0.42

Table 6: Operator $\times$ feasibility study: mean final cost $\pm$ seed std at $N = 100$ (10^4 SA steps, 10,000 instances, 3 seeds; all runs start from mean cost 57.87). Insertion with proactive masking wins and is also the most stable configuration.

Operator	Cost
Insertion	20.00
2-opt(*)	22.02
Swap	26.03

Feasibility	Cost
Masking	18.37
Indirect	20.36
Rejection	29.31

Table 7: Main effects: mean final cost of each factor level averaged over the other factor. Insertion and masking win marginally as well as jointly.

— visible in the swap column's large seed variance. Reactive rejection is uniformly the worst strategy for every operator: discarding infeasible proposals wastes both search budget and learning signal, and the policy must spend capacity learning the feasibility boundary instead of move quality. The indirect (greedy-split) representation is competitive in cost for insertion but pays the reconstruction at every step. The margin of the winning configuration over the runner-up (insertion + indirect, $+0.29$) is an order of magnitude above the insertion-row seed stds (0.03–0.04), and the runner-up shares the operator, so the operator choice stands even if the two leading feasibility strategies were tied.

Compute. Table 8 reports the evaluation wall-clock of every configuration. Quality and compute are essentially uncorrelated over the configurations — the cheapest column (rejection) is the worst on quality, and the most expensive configuration (2-opt(*) under masking) still loses to insertion — so the quality ranking of the main text is not a compute artefact.

Per-configuration diagnostics. Figures 5–7 give three complementary views of the set of configurations: the cost heatmap, the cost–compute trade-off, and the per-seed spread of the four best configurations. Seed variance grows quickly as configuration quality degrades, so the winning configuration is also the most reproducible one.

C Feature Ablation, Full Results

This appendix collects the protocol, complete plots, and secondary tables of the feature study behind the "Node representation" paragraph and Table 2, in the order of the main-text narrative: single-group diagnostics, thematic (cluster) removals, the head-to-head validation of the candidate best sets, the targeted single-group refinements, and the conditional pair descriptors. In the plot labels, the internal group names map to the paper terminology of Table 5 as `neighbor_rank` = neighbor distance-ranks,

Operator	Masking	Indirect	Rejection
Insertion	173	172	152
Swap	170	167	148
2-opt(*)	301	177	157

Table 8: Mean wall-clock time (s) to evaluate the full 10,000-instance test set for 10^4 SA steps. Rejection is cheapest because infeasible proposals are simply discarded; 2-opt(*) under masking pays $1.7\times$ the compute of every other configuration for its suffix-exchange feasibility checks.

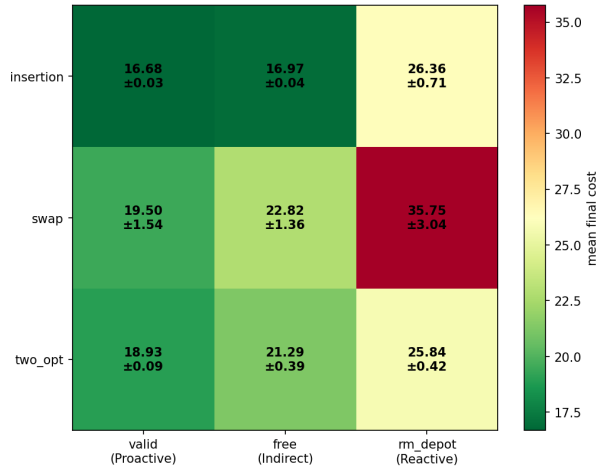

Figure 5: Mean final cost over the operator $\times$ feasibility set of configurations (green = better). The insertion/masking corner is the optimum; the rejection column is uniformly poor across all operators.

`demand_max` = relative demand, `density5/10%/33%` = $\mu_5/\mu_{N/10}/\mu_{N/3}$, `route_pct` = load ratio $L_{r(i)}$, `slack` = slack $S_{r(i)}$, and `node_pct` = load share η_i.

Protocol and reference points. The node representation is validated with three complementary protocols, all trained on the winning operator–feasibility pairing. *Leave-one-out* removes a single group from the full set and measures the degradation; *thematic* ablations remove a whole correlated cluster at once (e.g., all three density scales, or all capacity signals), exposing groups whose members are individually redundant but collectively useful; *isolation* adds a single group to a minimal base containing only the static and meta features, measuring its standalone value. From these measurements, candidate minimal feature sets are constructed — keeping groups whose removal hurts, dropping those that fail both the leave-one-out and the isolation test, and keeping one representative per correlated cluster — and confirmed seed-paired against the full set at 5 seeds; the conditional pair descriptors of the second stage are evaluated last, on top of the chosen set. The full 13-group representation reaches 16.678 ± 0.026, against 20.084 ± 0.049 for the static+meta base: the solution-dependent features cut cost by 3.41 routing units ($\approx 17\%$). The pooled 3-seed floor of the diagnostic sweep is $\sigma \approx 0.18$, inflated by the single high-variance no-

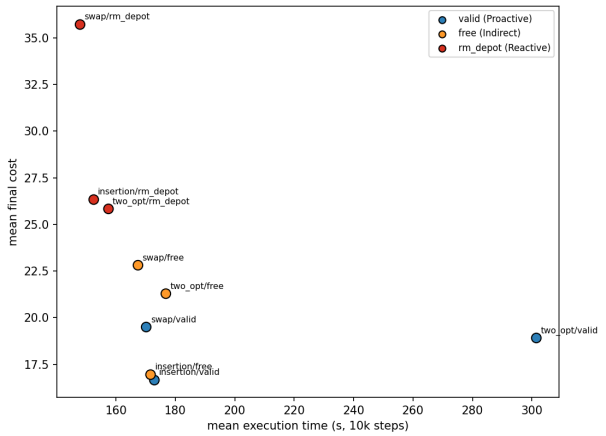

Figure 6: Cost against evaluation time for the nine configurations. Insertion/masking sits at the bottom left (best on both axes); 2-opt(*)/masking spends 1.7× the compute and remains worse.

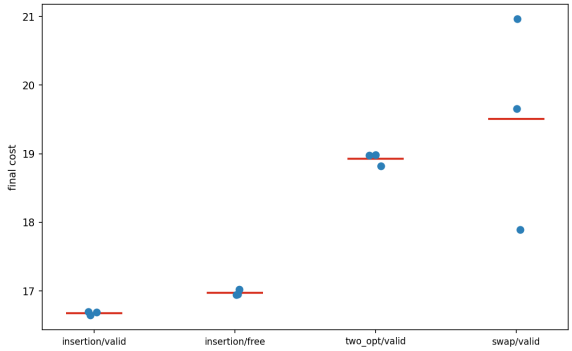

Figure 7: Per-seed final cost for the four best configurations. The winning configuration is tightly clustered; seed variance grows quickly as quality drops.

static configuration (typical seed std ≈ 0.05).

Single-group diagnostics. Table 9 reports the leave-one-out and isolation measurements. Four groups clear the noise floor at the margin: static (whose removal is catastrophic, $+3.84$), detour, topology, and meta. In isolation, the geometric and relational signals carry almost all the standalone value — detour ($+2.60$), topology ($+2.19$), neighbor ranks ($+1.26$), centroid ($+0.48$) — while the three capacity signals *hurt* when added alone to the base, and no density scale helps on its own. Applied mechanically, the single-feature pruning rule would therefore retain only six groups (static, meta, topology, neighbor ranks, detour, centroid) and drop the density, capacity, and relative-demand groups — the rule that the best-set validation below overturns. Figure 8 ranks all 30 ablation configurations and makes the two-cluster structure visible: leave-one-out runs stay near the full-set reference while isolation runs stay near the base, i.e., no single group is responsible for the 17% gap between the two. Figure 9 shows the per-seed spread of the reference configurations and explains why the pooled 3-seed floor is inflated by the single high-variance no-static run.

Group	Δ_{LOO}	Δ_{iso}
Static	$\mathbf{+3.84 \pm 0.85}$	(in base)
Detour Δ_i	$\mathbf{+0.60 \pm 0.08}$	$\mathbf{+2.60 \pm 0.07}$
Topology	$\mathbf{+0.46 \pm 0.08}$	$\mathbf{+2.19 \pm 0.15}$
Meta	$\mathbf{+0.31 \pm 0.08}$	(in base)
Slack $S_{r(i)}$	$+0.12 \pm 0.04$	-0.42 ± 0.31
Centroid $d_{i,\text{cent}}$	$+0.10 \pm 0.08$	$\mathbf{+0.48 \pm 0.06}$
Density $\mu_{N/3}$	$+0.09 \pm 0.12$	-0.08 ± 0.06
Neighbor ranks	$+0.03 \pm 0.05$	$\mathbf{+1.26 \pm 0.09}$
Relative demand	$+0.03 \pm 0.09$	-0.01 ± 0.02
Density μ_5	$+0.01 \pm 0.01$	-0.02 ± 0.09
Load share η_i	$+0.01 \pm 0.04$	-0.42 ± 0.10
Density $\mu_{N/10}$	-0.03 ± 0.07	-0.09 ± 0.09
Load ratio $L_{r(i)}$	-0.06 ± 0.08	-0.44 ± 0.36

Table 9: Single-group signals, sorted by marginal value. Δ_{LOO}: cost increase when the group is removed from the full set (positive = useful at the margin). Δ_{iso}: cost reduction when the group alone is added to the static+meta base (positive = useful standalone). Bold: exceeds the noise floor. Static and meta belong to the isolation base and have no standalone entry.

Theme removed	**Cost**	**Δ vs. full**
Relational (topology + nbr. ranks)	17.277	$+0.599 \pm 0.016$
Geometry (detour + centroid)	17.274	$+0.596 \pm 0.033$
Capacity (L, S, η)	17.021	$+0.343 \pm 0.059$
Density (μ_5, $\mu_{N/10}$, $\mu_{N/3}$)	16.822	$+0.145 \pm 0.033$

Table 10: Thematic ablation (3 seeds, seed-paired vs. the full set; positive Δ = the theme helps). The relational and geometry clusters are the two most valuable; capacity clears the typical seed noise while density does not.

Thematic ablation. Table 10 and Figure 10 report the removal of whole correlated clusters. This is the protocol that exposes collective value invisible to leave-one-out: no single capacity feature clears the floor on its own, but removing all three costs $+0.343$.

Best-set validation. The candidate sets of Table 2 were retrained head-to-head at 5 seeds on a batch with a clean floor ($\sigma_{\text{floor}} \approx 0.057$; per-set seed std between 0.04 and 0.07). Seed-paired against the full set, the pruned sets lose $+0.337 \pm 0.091$ (set7), $+0.405 \pm 0.077$ (nodepct6), $+0.417 \pm 0.049$ (centroid6), and $+0.479 \pm 0.075$ (core5) — every candidate significantly worse, degrading monotonically as groups are removed — while dropping the load ratio alone is a statistical tie (-0.035 ± 0.106). Figure 11 gives the per-seed view of the confirmation: the left panel shows that the 12-group and full-set clusters overlap and sit cleanly below every pruned candidate, and the right panel shows the seed-paired deltas against the noise floor — the evidence behind the reversal of the single-group pruning rule.

Set	Groups	Cost	Time (s)	Δ vs. full
full set	13	16.678	179	ref
− load ratio	12	16.619	166	-0.059 ± 0.076
− load ratio, − ranks	11	16.819	154	$+0.141 \pm 0.022$

Table 11: Targeted single-group refinements (3 seeds, seed-paired; mini-batch noise floor ≈ 0.035; per-set seed std between 0.02 and 0.05). Dropping the load ratio alone is a statistical tie with the full set; additionally dropping the neighbor ranks costs an incremental $+0.200 \pm 0.066$. The apparent 7% speedup of the 12-group set did not replicate at 5 seeds (Table 2) and is machine-load noise.

Targeted refinements. Table 11 and Figure 12 isolate the two borderline groups. Dropping the load ratio alone is a tie with the full set (reproduced independently at 3 and 5 seeds); additionally dropping the neighbor ranks costs an incremental $+0.200 \pm 0.066$, several times the floor — the neighbor ranks, not the load ratio, are the group that must stay.

Conditional pair descriptors. On top of the retained set, the two second-stage conditioning signals (edge distance-ranks and exact insertion cost, defined in Appendix A.1) were swept standalone and as a 2×2 cross-product (3 seeds, noise floor $\sigma \approx 0.071$). Both *hurt*: enabling the distance-rank descriptor costs $+0.14 \pm 0.11$ and the insertion-cost descriptor $+0.18 \pm 0.07$, each adding ≈ 20 s of wall-clock; the 2×2 main effects match the standalone deltas ($+0.13$ and $+0.17$) and the interaction is negligible (-0.02, within the floor), so the effects are additive and the 'both-on' configuration is the worst (Figure 13). The extra signal apparently distracts more than it informs: the per-node features already encode the local geometry the descriptors summarize.

D Architecture Study, Full Results

This appendix collects the candidate definitions, complete set of configurationss, and per-seed diagnostics of the architecture study behind the "Policy and learning" paragraph of the main text, one paragraph per design dimension in the order they were swept (critic, architecture, global context, shared encoder, distribution shaping), each sweep inheriting the previous winner. In the plot labels, the internal names map to the paper terminology as `deepsets` = multi-statistic critic, `ff` = mean-pooled feed-forward critic, `seq` = the reference actor with independent per-stage scorers, `shared` = the shared-encoder variant, `gc` = pooled global context, `bil` = bilinear second-stage head (0 = concatenation), and networks are written width/depth, e.g. `64/2L`.

Critic. The baseline critic scores each node with a small MLP and averages the scalar outputs into the value estimate, which makes V_ϕ a linear functional of the mean node representation. The alternative is the multi-statistic variant of the training section: nodes are encoded into embeddings, pooled with mean, max, and standard deviation, and mapped to the value by a nonlinear head — so dispersion information (clustered against scattered instances) survives the pooling.

Critic	Cost
multi-statistic	16.635 ± 0.038
feed-forward	16.693 ± 0.006

Table 12: Critic comparison (3 runs per arm; noise floor ≈ 0.027). The multi-statistic critic improves the mean final cost by 0.058, about twice the floor. The critic is used only during training, so inference cost is unchanged.

Net	Cost	Time (s)	Δ vs. 32×1
64×2	16.589 ± 0.004	238	-0.049 ± 0.039
64×1	16.591 ± 0.084	207	-0.047 ± 0.060
32×2	16.601 ± 0.008	187	-0.037 ± 0.030
32×1	16.637 ± 0.036	173	ref
16×2	16.649 ± 0.083	165	$+0.011 \pm 0.093$
16×1	16.668 ± 0.042	160	$+0.031 \pm 0.063$

Table 13: Actor architecture set of configurations (embedding width × hidden layers), sorted by mean cost; Δ is seed-paired against the 32×1 reference. No configuration clears the noise floor (≈ 0.053); the width main effect spans 0.069 against 0.019 for depth.

Table 12 compares the two. The critic only shapes the advantage estimates during training and is discarded afterwards, so the comparison is on solution quality alone.

Actor architecture. The embedding width and the number of hidden layers of the two scoring networks are swept jointly, trading representational ability against per-step inference cost. Table 13 gives the full width × depth configurations: the full spread from the smallest to the largest network is 0.079, no configuration clears the floor (≈ 0.053) against the 32×1 reference, and wall-clock rises by up to 38%; the faint monotone trend favors width over depth (main-effect spans 0.069 against 0.019). Figure 14 adds the seed-paired deltas against the noise band — every configuration falls inside it — and Figure 15 shows the cost–compute view: quality improves only marginally along a steeply rising compute axis, the signature of a capacity plateau.

Global context. The *global context* option pools the node representations into a summary g (mean, max, and standard deviation) appended to the input of both stages — the cheapest way to give the per-node scores access to instance-level information, at one pooling operation per step. It backfires: the mean cost rises from 16.633 to 17.143 (seed-paired $+0.508 \pm 0.496$, one seed degrading by $+1.08$) and the evaluation wall-clock from 180 to 255 s ($+48\%$) — the instance-level summary distracts the per-node scores rather than informing them. Figure 16 shows the per-seed effect: every seed degrades, one dramatically, so the regression is not a variance artefact.

Shared encoder. Figure 17 illustrates the variant: each node is encoded once into an embedding $\mathbf{h}_i$, and both selection stages are lightweight heads over the shared embeddings, the conditional score being computed either by a concatenation head over $[\mathbf{h}_j \mid \mathbf{h}_i]$ or as a bilinear compatibility

Stage-2 head	Context	Cost	Time (s)	Δ vs. ref
bilinear	on	19.830	196	$+3.196 \pm 0.103$
bilinear	off	19.860	168	$+3.226 \pm 0.163$
concat	off	20.154	192	$+3.520 \pm 0.251$
concat	on	20.160	236	$+3.526 \pm 0.099$

Table 14: Shared-encoder set of configurations, sorted by mean cost; Δ is seed-paired against the reference actor re-run with context off (16.633 ± 0.036, 180 s). Per-configuration seed std between 0.06 and 0.26; noise floor ≈ 0.176. Within the shared family the bilinear head beats concatenation (main effect -0.31) and the context toggle is neutral (-0.01), but every configuration is ≈ 3 units worse than the reference.

Clipping	Temp.	Cost	Time (s)	Δ vs. ref
$C = 10$	fixed	**16.457**	177	-0.173 ± 0.059
$C = 10$	learned	16.481	179	-0.149 ± 0.085
off	learned	16.580	176	-0.050 ± 0.069
off	fixed	16.630	175	ref

Table 15: Distribution-shaping set of configurations, sorted by mean cost; Δ is seed-paired against the both-off reference. Per-configuration seed std between 0.04 and 0.11; noise floor ≈ 0.072. Only logit clipping clears the floor; the learnable temperature does not help it (clipping main effect -0.136, temperature -0.014).

$\mathbf{q}^\top \mathbf{k}/\sqrt{d}$ between a query built from the selected node and a key per candidate. Table 14 and Figure 18 report its 2×2 set of configurations (head type $\times$ context) against the reference actor: every configuration sits $+3.20$ to $+3.53$ above it — a 19% quality loss, more than $18\times$ that batch's noise floor — so forcing both stages through one trunk destroys the pair-scoring ability at this problem size; the failure is structural rather than a bad corner of the set of configurations.

Distribution shaping. Two flags tame the proposal distribution without adding performance: *logit clipping*, which bounds the logits to $C\tanh(z)$ before the softmax so the distribution cannot collapse to a near-deterministic choice (Kool, van Hoof, and Welling 2019), and a *learnable softmax temperature* per stage. Table 15 and Figure 19 report their 2×2 set of configurations: only clipping clears the floor (all three seeds improve), while the learnable temperature is within noise on its own (-0.050 ± 0.069) and slightly degrades the clipped configuration when stacked, with doubled seed variance — one more trainable knob for PPO to destabilize.

E Search-Loop Components, Full Results

This appendix collects the full rankings and per-seed diagnostics behind the reward, annealing, acceptance, and initialization studies of the main text, one paragraph per design dimension. All four batches follow the standard protocol (3 seeds, 10,000 SA steps at half precision on the Nazari $N = 100$ test set); the reference configuration reproduces at 16.459–16.460 across the four batches, matching the adopted configuration of the architecture study (Table 15).

Reward. The reward decides which behavior PPO reinforces: per-step improvement, new records, or only the final outcome. Because the SA loop already owns part of the exploration — worsening moves are accepted by the Metropolis criterion, not chosen by the policy — it is not obvious how much of the trajectory should be credited to the actor. Let Δ_t denote the cost improvement realized by the executed transition (zero when the move is rejected) and B_t the best cost found so far. The compared variants are: *immediate*, $r_t = \Delta_t$ with a fixed penalty for infeasible proposals; *acceptance-aligned*, which rewards accepted improving moves by Δ_t but deliberately assigns zero to accepted worsening moves (exploration is SA's responsibility, not the policy's fault) and a small constant penalty to rejected proposals; *best-so-far*, $r_t = \max(0, B_{t-1} - B_t)$, which only rewards new records; *terminal*, a single end-of-episode reward equal to the relative improvement over the initial cost; *mixtures*, either a fixed convex combination of the immediate and best-so-far signals or a scheduled interpolation that starts from the dense acceptance-aligned signal early in training and shifts toward the sparser best-so-far or terminal signals; and *step-penalty*, a negative per-step reward proportional to the current best cost, pressuring fast descent. In the plot labels, the internal reward names map to this terminology as `immediate` = immediate, `sa_aligned` = acceptance-aligned, `global_best` = best-so-far, `hybrid` = the fixed mixture ($\alpha = 0.5$), `curriculum` and `curriculum_terminal` = the scheduled interpolations toward best-so-far and terminal respectively, and `step_penalty_init` and `step_penalty_16` = the per-step penalty normalized by the initial cost and by a fixed constant close to the typical final cost.

Table 16 splits the nine candidates into two regimes (noise floor 0.042, the pooled seed std of the stable cluster). Every reward built on dense per-step improvement lands in a band only 0.07 wide: the reference immediate reward is best at 16.459, the fixed mixture is a statistical tie (seed-paired $+0.001 \pm 0.042$), and the acceptance-aligned, best-so-far, and scheduled variants trail by at most 0.07 — the actor is largely insensitive to the exact shaping as long as the signal is dense. Removing the per-step credit is catastrophic: the terminal reward loses 2.95 routing units with seed swings of almost a full unit, and the step-penalty schemes lose 0.7 to 1.1 with similarly unstable training — the penalty fights the improvement signal instead of complementing it. Figure 20 zooms on the stable cluster and shows the seed-paired deltas against the noise band.

Cooling schedule. The cooling schedule controls how quickly the acceptance criterion hardens from tolerant exploration to strict descent; a learned proposal could in principle prefer a different cooling profile than blind SA. The *shape* comparison holds the temperature range (T_0, T_{final}) fixed and opposes the *geometric* decay $T_{k+1} = \alpha T_k$ to a piecewise-constant *step* schedule that drops the temperature at one half, two thirds, and three quarters of the horizon. It is a statistical tie: 16.460 against 16.473, a seed-paired -0.013 ± 0.034 against a noise floor of 0.044, with the per-seed sign split

Reward	Cost
immediate (reference)	**16.459**
fixed mixture	16.460
sched. → best-so-far	16.503
best-so-far	16.510
acceptance-aligned	16.516
sched. → terminal	16.531
step-penalty (const.)	17.198
step-penalty (init)	17.549
terminal	19.411

Table 16: Reward ablation: mean final cost over 3 seeds, sorted; the rule separates the dense stable cluster from the sparse and penalty-based rewards. Wall-clock is flat across arms (173–177 s). Per-seed diagnostics in Figure 20.

$T_0 \backslash T_{\text{final}}$	**0.01**	**0.1**
0.5	16.342	16.475
1	16.343	**16.508**
2	16.354	16.486

Table 17: Temperature-rangeset of configurations: mean final cost over 3 seeds; rows = initial temperature T_0, columns = final temperature T_{final} (reference in bold).

and identical wall-clock (Figure 21). The *range*, by contrast, holds the largest single-knob improvement of the whole validation: a 3×2 set of configurations retrains the policy over $T_0 \in \{0.5, 1, 2\}$ and $T_{\text{final}} \in \{0.01, 0.1\}$ (Table 17) and splits cleanly by final temperature — every $T_{\text{final}} = 0.01$ configuration beats every $T_{\text{final}} = 0.1$ configuration, and the reference range $(1, 0.1)$ is in fact the worst configuration. Cooling deeper gains -0.165 ± 0.034 at the reference T_0, four times the 0.041 floor, with a unanimous sign at every starting temperature; the initial temperature is inert (at $T_{\text{final}} = 0.01$ the three starting points are a three-way tie). Since 0.01 is the lowest tested value, an even lower floor cannot be ruled out.

Metropolis acceptance. Once the proposal is learned, does the stochastic acceptance still earn its keep — and should the policy be trained with it in the loop? A 2×2 ablation enables or disables the Metropolis criterion independently at training and at test time (the "off" arm accepts every proposed move), quantifying whether the learned proposal replaces the stochastic acceptance or complements it. The cross (Table 18) shows the criterion earns its keep at inference, not at training: removing the acceptance at test time costs $+0.119 \pm 0.025$ over the Metropolis-trained actor and $+0.094 \pm 0.009$ over the greedy-trained one — several times the 0.034 floor and unanimous across seeds — whereas the training-time flag is within noise at the standard test-on inference ($+0.027 \pm 0.045$, sign split). Only when deploying greedily does greedy training pay ($+0.052 \pm 0.050$), a mild train/test-consistency effect outside the regime we deploy in. Figure 22 decomposes the cross into its two effects; the greedy-trained arms also show markedly lower seed variance

Train \ Test	Metropolis	Accept-all
Metropolis (reference)	**16.459**	16.578
Accept-all	16.432	16.526

Table 18: Metropolis 2×2 cross: mean final cost over 3 seeds; rows = training setting, columns = inference setting. Per-seed effects in Figure 22.

(0.010/0.020 against 0.040/0.050).

Initialization. Every episode starts from a constructive solution, both during training and at inference, and the two roles need not agree: at training time the construction fixes the state distribution the policy learns from, while at inference it is a free knob of the solver. Training starts from a single constructive heuristic — random assignment, nearest neighbor, sweep, or single-customer routes — or from a progressively introduced mixture of all four; every trained policy is additionally cross-tested from all five test-time constructions (the four above plus Clarke–Wright savings). The heuristics appear under their code names `random` (random assignment), `sweep`, `nearest_neighbor`, `Clark_and_Wright` (Clarke–Wright savings), and `isolate` (single-customer routes); their construction costs on the test set are 57.9, 30.5, 19.0, 16.4, and 104.0 respectively. Table 19 gives the full train $\times$ test cross. At inference the construction barely matters — the random-trained actor lands in a band only 0.160 wide, the one real edge being a Clarke–Wright start, which already sits at the fixed point of the search (-0.153 ± 0.021 against the 0.063 stable floor). At training time it is decisive: actors trained from the structured heuristics lose 1.1 to 2.8 routing units at the standard inference, unanimously and far past even the global 0.395 floor, and mixing all four constructions into training dilutes coverage, a small but unanimous regression ($+0.222 \pm 0.084$). Figure 23 shows the cross as a heatmap, and Figure 24 shows the two slices that support the main-text reading: training construction matters, inference construction barely does, and the matched train/test diagonal — worse than the random-trained actor everywhere — rules out a mismatch explanation. The final model reproduces this test-time edge on the headline NeuOpt set: at $T = 10^4$ a Clarke–Wright start reaches 16.117 against 16.183 from a random start (the "C–W" row of Table 3), a 0.066 gain. But the head start is nearly all of it — Clarke–Wright constructs at 16.5 and the search removes only 0.4 from there, whereas from a random start it descends the full way from 57.9 — so the `C-W` column measures the quality of the construction, not extra work by the actor, and the edge is expected to shrink as the budget lengthens and the random start catches up.

F Optimization Hyperparameters, Full Results

This appendix documents the optimization-hyperparameter sweeps and their combined-configuration validation, in particular the reproducible divergence that fixed the final critic learning rate. Every design dimension before this one compared design alternatives; the plain numerical knobs of the

Train \ Test	Rand.	Sweep	NN	C–W	Isol.
random (reference)	**16.460**	16.467	16.467	16.307	16.444
sweep	17.548	16.950	16.854	16.271	17.148
nearest neighbor	19.228	17.729	17.401	16.381	18.837
single-customer	17.736	17.725	17.753	16.442	17.706
mixed	16.682	16.663	16.700	16.357	16.673

Table 19: Initialization cross: mean final cost over 3 seeds, rows = training construction, columns = test-time construction (NN = nearest neighbor, C–W = Clarke–Wright savings, Isol. = single-customer routes). The random-trained row is uniformly the best, and its spread across test constructions is only 0.160 although the construction costs span 16.4 (C–W) to 104.0 (single-customer). The structured-training rows recover only in the C–W column, where the construction itself already sits at the fixed point of the search — that column measures the inference, not the actor. Worst case over inferences: 16.467 (random), 16.700 (mixed), 17.5–19.2 (structured).

optimizer were inherited, not tuned. With the architecture and search loop frozen, the knobs are swept around the reference configuration one dimension at a time: the actor and critic learning rates (a 3×3 set of configurations), the entropy coefficient, and the rollout length — the number of SA steps collected per PPO update, which sets both the horizon over which advantages are estimated and how far into the cooling schedule the collected experience reaches. Every reference value lies strictly inside its configuration set, so each sweep is a proper local search around the default. All batches follow the standard protocol (3 seeds per configuration, 10,000 SA steps at half precision on the Nazari $N = 100$ test set); the per-batch noise floors are 0.031 (learning rates), 0.046 (entropy), 0.041 (rollout), and 0.041 (temperature range), and the reference configuration reproduces at 16.459–16.508 across the four batches. A fifth sweep, the number of PPO passes per collected batch, is run at the end of the chain on the configuration that all the above produce, and therefore carries its own floor and its own reference level.

Learning rates. The actor learning rate is the dimension that matters, and its effect is monotone (Table 20): the three worst configurations all share the highest actor rate, the three best all share the lowest, and each step increase in the actor learning rate costs roughly 0.05 on average across critic-rate settings. The critic rate is a weaker, second-order knob. The best configuration — actor 3×10^{-4}, critic 3×10^{-3} — improves on the reference by -0.064 ± 0.051, unanimous across seeds against the 0.031 floor; the combined-configuration runs below explain why the adopted configuration nevertheless keeps the critic rate at 10^{-3}.

Entropy and rollout. The entropy coefficient is a clean confirmation of the default (Table 21, top): 0.01 sits at the bottom of a U, with both neighbors unanimously worse — removing the bonus costs $+0.128 \pm 0.021$ (the policy under-explores and collapses onto a narrow move set), overweighting it costs $+0.783 \pm 0.036$ (near-uniform proposals). The rollout length is asymmetric (bottom): halving it to 50 is a large unanimous regression ($+0.444 \pm 0.015$) — the collected experience never reaches the cool end of the schedule where the hard moves live — while doubling it to 200 is a marginal, sign-split gain (-0.056 ± 0.053) at twice the collection cost per update.

Actor \ Critic	3×10^{-4}	10^{-3}	3×10^{-3}
3×10^{-4}	16.447	16.435	16.396
10^{-3}	16.509	**16.460**	16.466
3×10^{-3}	16.509	16.502	16.510

Table 20: Learning-rate set of configurations: mean final cost over 3 seeds; rows = actor rate, columns = critic rate (reference in bold).

Entropy coefficient	Cost
0	16.587
0.01 (reference)	**16.459**
0.05	17.242
Rollout length	
50	16.904
100 (reference)	**16.460**
200	16.404

Table 21: Entropy-coefficient and rollout-length sweeps: mean final cost over 3 seeds.

PPO passes per batch. The number of optimization passes the actor and critic take over each collected batch was held at 3 for every component sweep, to keep training cheap, and is tuned once at the end of the chain on the no-curriculum configuration (3 seeds per arm, 4 for the last one; noise floor 0.051, pooled over the stable arms). Table 22 shows the trade-off. The mean improves monotonically from 1 to 10 passes, for a total of 0.081, which is the expected shape: more gradient steps extract more signal from the same experience. But the seed spread travels with it — the 10-pass arm is three times as wide as the 5-pass arm, and its mean rests on two good seeds against a third that regresses — and the improvement it buys, -0.022 ± 0.093, is inside the floor and sign-split, hence a tie. One grid step further, at 15 passes, the updates are large enough to destabilize training outright: two of four seeds end at 19.57 and 20.10, roughly 23% above a converged solution, and the failure is frequent rather than anecdotal (the fourth seed was added precisely to test whether the first divergence was a one-off). Since the number of passes is a training cost only — inference is a fixed SA rollout and its wall-clock is flat across arms at 169–174 s — the decision is governed by stability and training time, and 5 dominates its only rival on both while matching it to within the noise.

Combined-configuration runs. The three wins — the lower actor rate, the longer rollout, and the deeper cooling floor of the annealing study — were each measured in isolation, so the joint configuration is retrained from scratch to test additivity. With the best critic rate of 3×10^{-3}, five

PPO passes	Cost	Std	Δ vs. 5
1	16.343	0.056	$+0.059 \pm 0.045$
3	16.319	0.023	$+0.036 \pm 0.023$
5 (adopted)	**16.284**	0.024	—
10	16.262	0.078	-0.022 ± 0.093
15	18.069	2.049	$+1.286 \pm 2.203$

Table 22: PPO passes per collected batch: mean final cost over 3 seeds (4 for the last arm), with seed-paired deltas against the adopted value on the shared seeds. Noise floor 0.051, pooled over the stable arms above the rule; the 15-pass arm is excluded from the floor because its spread is divergence, not sampling noise — two of its four seeds end at 19.57 and 20.10, and its converged-seed mean (16.305) is no better than the 3-pass arm.

Seed	Critic 3×10^{-3}	Critic 10^{-3}
0	16.265	16.257
1	19.865 / 19.872	16.268
2	16.219	16.186
3	16.258	16.209
4	16.152	—
5	16.186	—
mean (converged)	16.216 ± 0.048	16.230 ± 0.039

Table 23: Per-seed final cost of the combined configuration (actor rate 3×10^{-4}, rollout 200, $T_{\text{final}} = 0.01$) under the two candidate critic learning rates. Seed 1 under the 3×10^{-3} critic was trained twice independently; both runs diverge into the same degenerate basin. The 10^{-3} critic converges on every seed.

of six seeds land at 16.216 ± 0.048, matching the sum of the individual effects; reducing the critic rate to the reference 10^{-3} converges on every seed at 16.230 ± 0.039, a seed-paired -0.223 ± 0.016 against the reference configuration, unanimous — the adopted configuration. Two caveats carry over to the main text: the two winning values (actor rate and cooling floor) both sit on the boundary of their explored ranges, and the divergence below shows that stacking individually-validated winners can create interactions none of them exhibits alone. Table 23 lists every training run of the two combined configurations. The aggressive variant (critic rate 3×10^{-3}, the best configuration of the learning-rate sweep) was extended from 3 to 6 seeds after the first divergence, and the failing seed was retrained a second time from scratch: both runs land at 19.86–19.87, so the failure is deterministic given the seed, not a flaky run. The same seed trains without incident under the reference configuration (16.473) and under the final configuration (16.268), so the seed itself is not pathological — the divergence is created only by the combination of knobs, each of which was stable across all its seeds in isolation.

Failure mechanism. The quantity that overflows is not the learning rate itself but the product of learning rate and advantage, and it is the fast critic that manufactures the oversized advantages. With a 200-step rollout cooling to $T_{\text{final}} = 0.01$, the long low-temperature tail of every collected episode is a regime where the Metropolis rule rejects nearly every non-improving move; a rejected move leaves the state unchanged and its immediate reward is exactly zero, so early in training — when the near-random policy finds few improving moves — the return signal is mostly zeros punctuated by rare spikes. A critic stepping $10\times$ faster than the policy it evaluates overshoots when fitting that target, and an occasional badly mis-scaled value estimate produces one outsized advantage, hence one outsized policy step; the updated policy then jumps far enough from the collection policy that the exponentiated log-ratio overflows in the backward pass and the run never recovers. The seed dependence is a race: a seed that lingers longer in the flat zero-reward regime gives the fast critic more opportunities to overshoot before the policy finds signal — the failing seed spent roughly 50 epochs there where a converging seed escaped in about 5. Matching the critic rate to the reference 10^{-3} keeps the advantages bounded and closes the race at no measurable cost: the two variants are statistically indistinguishable on their common converged seeds (Table 23).

G Curriculum Study, Full Results

This appendix collects the complete curriculum results: the no-curriculum baseline and the noise floor it defines, the three mechanisms of Appendix A.7, the budget-matched coverage ablation, and the 5-seed confirmation that settles the choice. The whole study sits on the configuration frozen by all preceding design dimensions, with the curriculum as the only free component; it therefore has its own reference level and its own noise floor, and none of its numbers may be compared across batch boundaries. In the sweep configuration files, `CL_MODE=warmup/chain` selects the warm-up or the persistent-chain mechanism, `MAX_OUTER_STEPS_CL` is the warm-up strength ξ_{CL}, `MAX_PROB_STEP` the ramp duration E, `STEEP_SIG` the sigmoid steepness κ, `CL_RANDOM_DEPTH` the per-instance stochastic depth, and `CHAIN_REFRESH` the chain refresh rate ρ. Setting `CL_TYPE=lin` together with $E = 1$ disables the ramp, which is how the stochastic-depth arms isolate ξ_{CL} as their only knob. All arms are evaluated under the standard protocol (10,000 SA steps at half precision on the Nazari $N = 100$ test set, identical instances throughout, mean initial cost 57.87).

Baseline and noise floor. The no-curriculum baseline reaches 16.238 with a seed-to-seed standard deviation of 0.006 over 5 seeds (16.2277, 16.2356, 16.2405, 16.2411, 16.2437) — five seeds spanning 0.016 routing units end to end, which is itself a check that the frozen configuration trains reproducibly. The floor used throughout this appendix is the more conservative $\sigma_{\text{floor}} = 0.030$, the pooled within-configuration standard deviation of the baseline and the ramped warm-up; it is applied unchanged to every comparison below so that all verdicts are commensurable. Arms trained on 3 seeds are paired against the 3-seed subset of the baseline (16.2348), never against its 5-seed mean.

Mechanism	Cost	Std	Δ vs. none	Resp.
no curriculum	16.238	0.006	—	—
ramped warm-up	**16.165**	0.042	-0.073 ± 0.043	$4/5$
stochastic depth	16.204	0.015	-0.034 ± 0.015	$3/5$
persistent chains	16.487	0.053	$+0.253 \pm 0.047$	$0/3$

Table 24: The three curriculum mechanisms at their best setting, seed-paired against the no-curriculum baseline; 5 seeds except persistent chains (3 seeds, paired against the matching baseline subset). *Resp.* counts the seeds that individually improve past the floor $\sigma_{\text{floor}} = 0.030$. The ramped warm-up uses $\xi_{\text{CL}} = 250$, $E = 2000$, $\kappa = 0.01$; the stochastic depth $\xi_{\text{CL}} = 250$ with the ramp disabled; the chains $\rho = 0.2$.

The ramped warm-up survives, on four seeds out of five. The warm-up ramp improves the baseline by -0.073 ± 0.043, about 2.4 times the floor, so the mechanism is emphatically not a null (Table 24). The effect is nonetheless split: four seeds improve by 0.079 to 0.106, while one moves by $+0.002$, statistically identical to its baseline value. That single non-responder is what inflates the arm's spread (0.042 against the baseline's 0.006) and breaks unanimity, and the same seed reappears in every split verdict of this study.

Both alternatives are monotone in pre-optimization depth. The stochastic depth is strictly monotone in ξ_{CL} and only its shallowest arm improves on the baseline; the chains are monotone in ρ, i.e. they improve as the mechanism is used *less*, and their limit $\rho \to 1$ is the baseline by construction. Table 25 places all six arms on the single axis they share, the mean number of SA steps by which the trajectory has been advanced before the actor collects experience. The reading is consistent: the shallower the pre-optimization, the better the policy — the actor learns from states it can still improve, and advancing the trajectory past a few hundred steps starves it of the reward signal it trains on, compounding the long zero-reward tail that the deep cooling floor already produces. Two caveats block reading the table as one curve. The two families are disjoint in depth (125–500 against 1000–4000), so mechanism and depth are confounded; and at the boundary the ordering inverts, the deepest warm-up arm being worse than a chain arm at twice its depth. Persistent chains are therefore rejected on their own evidence — nine runs, all significantly worse than doing nothing, with the sweep's own gradient pointing back at the baseline — rather than as a corollary of a depth law.

Coverage is a mean-depth effect, not a mixing effect. The advantage of the stochastic depth could be the depth *mix* — different instances warmed to different depths within one batch — or simply its lower mean depth. Two constant-depth controls separate the two (Table 26). At matched loop length, hence matched wall-clock, the mix is worth -0.261 against a constant warm-up to $\xi_{\text{CL}} = 250$, unanimously and about nine times the floor; and that constant arm is not merely worse than the mix but $+0.224$ *above the baseline*, i.e. actively harmful, exactly the over-optimization the monotonicity above predicts. The sharper control is the constant warm-up at the mix's own effective depth, and there the edge collapses to -0.016, below the floor and split across seeds. Mean depth is what matters; mixing per se is a wash at the mean. Its one measurable effect is local: the entire gap against the matched-depth control comes from the same hard seed that the ramp cannot move, which the mix pulls from 16.330 to 16.190 while leaving the other seeds marginally worse.

Mechanism	Setting	Depth	Δ vs. none
stochastic depth	$\xi_{\text{CL}} = 250$	125	-0.037 ± 0.011
stochastic depth	$\xi_{\text{CL}} = 500$	250	$+0.135 \pm 0.037$
stochastic depth	$\xi_{\text{CL}} = 1000$	500	$+0.302 \pm 0.033$
persistent chains	$\rho = 0.20$	1000	$+0.253 \pm 0.047$
persistent chains	$\rho = 0.10$	2000	$+0.390 \pm 0.042$
persistent chains	$\rho = 0.05$	4000	$+0.523 \pm 0.084$

Table 25: All stochastic-depth and persistent-chain arms (3 seeds each, 18 runs), ordered by effective pre-optimization depth in SA steps ($\xi_{\text{CL}}/2$ for the warm-up, $\text{OUTER\_STEPS}/\rho$ for the chains, with $\text{OUTER\_STEPS} = 200$). Deltas are seed-paired against the 3-seed baseline subset; every arm is unanimous across its seeds. Only the shallowest arm improves on no curriculum at all.

Warm-up	Depth	Cost	Δ vs. none
constant, $\xi_{\text{CL}} = 250$	250	16.459	$+0.224$
stochastic, $\xi_{\text{CL}} = 250$	125	**16.198**	-0.037
constant, $\xi_{\text{CL}} = 125$	125	16.214	-0.020

Table 26: Coverage ablation, 3 seeds per arm, seed-paired against the baseline subset (16.2348); *Depth* is the mean number of warm-up SA steps. The first row matches the stochastic arm's loop length (and wall-clock, since freezing an instance does not shorten the loop), the third its effective depth. All deltas are unanimous except the last, which is split and below the floor.

Confirmation. The two surviving mechanisms are compared head to head on all 5 seeds, the decision rule fixed in advance being to prefer the simpler stochastic depth only if it wins or ties unanimously (Table 27). It does neither: the ramp leads by 0.039 ± 0.053 on the mean, above the crude floor, and 4 of the 5 seeds prefer it — the exception being, once more, the seed the ramp fails to move. Two qualifications cut against over-reading the margin. The paired 95% confidence interval, $[-0.027, +0.105]$, includes zero, so by a strict test the two are tied; but the tie is not symmetric, since going from 3 to 5 seeds moved the point estimate from 0.018 to 0.039, i.e. further toward the ramp, the two added seeds behaving like its responders rather than like the hard seed. The ramped warm-up is adopted. The stochastic depth remains the simpler and more per-seed-uniform runner-up — two fewer hyperparameters, no ramp duration and no steepness — and the one regime in which it might still pay off, out-of-distribution evaluation, is untested here.

Seed	Ramp	Stoch. depth	Δ
0	16.237	**16.190**	-0.047
1	**16.162**	16.201	$+0.039$
2	**16.142**	16.204	$+0.062$
3	**16.135**	16.229	$+0.095$
4	**16.148**	16.196	$+0.047$
mean	$\mathbf{16.165} \pm 0.042$	16.204 ± 0.015	$+0.039 \pm 0.053$

Table 27: Five-seed confirmation, ramped warm-up against stochastic depth at $\xi_{\mathrm{CL}} = 250$; Δ is stochastic $-$ ramp, so a positive value favors the ramp. Bold marks the better configuration on each seed. Seed 0 is the recurring hard seed that the ramp does not move and that the depth mix rescues.

H Generalization and Scalability, Full Results

This appendix collects the measurements behind Section 5.4: the step-budget sweep that fixes the quality/time trade-off, the dimension sweep beyond the training size, and the two implementation sweeps — batch size and numerical precision — that determine how every other evaluation in the paper is run. All of them use the single final checkpoint, trained at $N = 100$ with $Q = 50$, without retraining.

Protocol and comparability. Unlike the ablation batches, these sweeps run on our internally generated Nazari test sets (`gen_nazari_{N}.pt`) rather than on the NeuOpt test set used in Table 3, and in `float32` rather than `float16` unless stated otherwise. The two draws are statistically equivalent but not identical (mean constructive cost 57.86 against 57.90 at $N = 100$), so costs here should be read against each other and not transferred into Table 3; the agreement is nevertheless close (15.899 here against 15.923 there at $T = 10^5$, $N = 100$). Reported times are search time only: instance construction — which materializes a dense $[N{+}1, N{+}1]$ distance matrix and its argsort, and reaches 1.34 s at $N = 10{,}000$ — is timed separately and excluded throughout.

Step budget. Table 28 sweeps the improvement budget from 10^2 to 10^6 steps. The two blocks use different batch sizes (10,000 instances up to $T = 10^4$, 1,000 beyond, for memory reasons), so times are comparable *within* a block only, and costs across blocks come from different subsets of the same distribution. Returns diminish sharply and regularly: at $N = 100$, the first decade ($10^2 \to 10^3$) buys 6.68 routing units, the next 0.79, then 0.21, 0.05 and finally 0.11 over the last two decades combined. Reaching 15.788 at $T = 10^6$ costs $100\times$ the compute of 16.118 at $T = 10^4$ for a 2% improvement, which is the regime where the conventional solvers of Table 3 remain out of reach. A second effect is visible in the lower block: at $N \leq 50$ the run time is essentially independent of N (≈ 111 s for 10^5 steps at $N = 10$, 20 and 50), because a batch of 1,000 small instances does not saturate the GPU and the step count alone sets the cost.

Guided against blind search. Figure 25 gives the anytime frontier of both searches at $N = 100$, on the test set and in the precision of Table 3. Every point is an *independent, complete* run: the temperature schedule is compressed to that point's budget, so a 10^3-step run is a fully annealed search and not a prefix of the 10^5-step one. This is the only construction under which the two methods can be compared at a budget, and it is what a single long trace cannot provide — sampled along one $2 \cdot 10^5$-step anneal, blind SA reads 53.99 at step 10^4, against 22.99 for a blind run actually budgeted at 10^4 steps, because the first is still mid-anneal. The frontier reproduces the "Ours" block of Table 3 to three decimals (16.185 and 22.989 at $T = 10^4$; 15.923 and 18.130 at $T = 10^5$), which is the intended cross-check: the figure and the headline table are one measurement.

	$N = 10$		$N = 20$		$N = 50$		$N = 100$	
T	Cost	s	Cost	s	Cost	s	Cost	s
batch $= 10{,}000$ *instances*								
10^2	4.796	0.2	6.673	0.3	12.469	1.1	23.588	2.4
10^3	4.644	1.6	6.354	3.4	10.978	11	16.911	23
$5 \cdot 10^3$	4.590	7.8	6.257	17	10.717	56	16.270	115
10^4	4.573	16	6.227	34	10.643	113	16.118	230
batch $= 1{,}000$ *instances*								
10^5	4.507	111	6.157	111	10.517	113	15.899	156
$2 \cdot 10^5$	4.498	222	6.144	222	10.484	223	15.853	313
$5 \cdot 10^5$	—	—	—	—	10.457	562	15.813	770
10^6	—	—	—	—	10.434	1121	15.788	1530

Table 28: Mean cost and total search time against the improvement budget T, `float32` on one GPU. Times are totals over the whole batch and are comparable only within a block; the two blocks use different batch sizes and therefore different instance subsets.

Panel (a) shows that the guided search is better than the blind one at every budget, and that the two do not merely differ by a constant: LG-SA is 27.7 ahead at $T = 10^2$, 19.5 at 10^3, 6.8 at 10^4 and 2.2 at 10^5, so the blind search is slowly closing a gap it never closes. What the policy buys is position on the curve rather than a different asymptote — both methods are drifting toward the same annealing limit, and guidance moves the search there roughly two orders of magnitude sooner. LG-SA needs 300 steps to pass blind SA's 10^4-step result and 10^3 to pass its 10^5-step result — in both cases about two decades of budget.

Panel (b) re-plots the same runs against wall clock and is the comparison that matters, since a guided step is not a blind step. The per-step overhead of the actor is 13.1 to $13.7\times$ across the whole sweep — remarkably stable, and measured here rather than inferred from two totals as in earlier versions of this comparison. Even after paying it, the guided curve dominates over the entire range where both are measured: the best point blind SA reaches at all, 18.130 after 139 s, is already beaten by LG-SA after 17.7 s (16.982) — better quality in $7.9\times$ less time — and by 1.65 at 52.9 s. At matched wall clock near the paper's operating point, LG-SA's 16.185 in 176 s stands against blind SA's 18.130 in 139 s. The margin narrows as both curves flatten toward HGS, and the crossing point — if the blind search has one within a practical budget

		$T = 10^3$			$T = 10^4$	
N	Init.	Cost	Improv.	ms/step	Cost	Improv.
100	59.4	19.1	-68%	0.46	18.3	-69%
200	119.2	34.5	-71%	0.51	31.3	-74%
500	288.8	89.5	-69%	0.62	75.3	-74%
1,000	588.5	215.0	-63%	0.79	147.9	-75%
2,000	1188.4	599.0	-50%	1.09	309.4	-74%
5,000	2944.6	2164.7	-26%	1.91	850.3	-71%
10,000	5866.4	5022.1	-14%	3.32	2100.6	-64%

Table 29: Sequential single-instance scaling, CPU (12 threads), batch 1, `float32`, mean over 3 runs. Instances are drawn from the Nazari distribution with capacity fixed at the training value $Q = 50$. The ms/step column is measured at $T = 10^4$ and agrees with the $T = 10^3$ measurement to within 2% at every size.

— lies beyond 10^5 steps and is not established by these measurements.

Dimension. Table 29 gives the full version of Table 4, with both step budgets. Read row-wise, it separates the two things that change with N. The per-step cost is nearly flat: 0.46 ms at $N = 100$ against 3.32 ms at $N = 10{,}000$, a $7.2\times$ increase for $100\times$ the nodes, and it is the same at 10^3 and 10^4 steps, confirming that it measures the step and not the schedule. Solution quality, on the other hand, is governed by steps per node. At 10^3 steps the improvement over the constructive solution decays monotonically with size, from -68% at $N = 100$ to -14% at $N = 10{,}000$; multiplying the budget by ten restores it to -64% at $N = 10{,}000$ and leaves the -69% at $N = 100$ almost unchanged, since that size was already budget-saturated. The residual dip at the two largest sizes is consistent with 10^4 steps being only one to two proposed moves per node.

Batch size. Because the search is a fixed-length loop of batched tensor operations, the GPU can amortize it over many instances at once, and Table 30 locates the operating point. Throughput is maximal at a batch of $1{,}000$ instances (15.0 ms per instance, 67 instances per second) and then *degrades* by about 45% to a plateau of ≈ 22 ms per instance from batch $4{,}000$ on, where the kernels become memory-bound; the small-batch end is worse still (109 ms at batch 100), dominated by per-step launch overhead. Peak memory is linear in the batch (0.19 MB per instance at $N = 100$), so the plateau, not memory, is the binding constraint. Solution quality is flat across the whole range once the sample is large enough to be stable, as it must be — the batch dimension is independent in the solver. We nevertheless evaluate at batch $10{,}000$ everywhere in the paper, accepting the 45% throughput loss, so that every reported cost is a mean over the same full $10{,}000$-instance test set.

Numerical precision. Table 31 compares the three inference precisions. Half precision is free: `float16` is $1.29\times$ faster than `float32` for a cost difference of -0.0002, four orders of magnitude below the noise floor of any comparison in this paper. `bfloat16` runs at the same speed but loses

Batch	Cost	Total (s)	ms/inst.	Peak mem. (MB)
100	16.403	10.9	109.2	29
500	16.126	11.1	22.2	105
1,000	16.162	15.0	**15.0**	198
2,000	16.188	31.6	15.8	388
3,000	16.148	54.5	18.2	570
4,000	16.130	87.8	21.9	758
5,000	16.118	109.0	21.8	943
6,000	16.115	130.4	21.7	1129
8,000	16.107	175.3	21.9	1507
10,000	16.125	220.3	22.0	1877

Table 30: Effect of the evaluation batch size on GPU throughput and memory, at $N = 100$ and $T = 10^4$, `float32`. Rows at $7{,}000$ and $9{,}000$ are omitted; they interpolate the plateau. The cost column varies only through the instance sample, which is the first B instances of the test set.

Precision	Cost	Δ	Time	Speedup
`float32`	16.1213	—	3m40s	$1.00\times$
`float16`	16.1211	-0.0002	2m51s	$1.29\times$
`bfloat16`	16.1867	$+0.0654$	2m51s	$1.29\times$

Table 31: Inference precision, at $N = 100$, $T = 10^4$, batch $10{,}000$. Differences are relative to the `float32` baseline.

0.065 — twice the ablation noise floor, and enough to change the verdict of several design comparisons. The two formats have the same width and differ only in the exponent/mantissa split, so this is a mantissa effect: the policy scores node pairs through small differences between comparable distances, and `bfloat16`'s 8 mantissa bits are too coarse to rank them reliably, whereas its wider exponent range buys nothing on inputs normalized to $[0, 1]$. All evaluations in the paper accordingly use `float16`. Note that this concerns inference only; training is unchanged.

Structured out-of-distribution instances (XML100). Every result so far is measured on the Nazari distribution the model was trained on. The XML100 benchmark of Queiroga et al. (2022) probes the opposite regime: $10{,}000$ heterogeneous 100-customer instances generated with the Set X methodology — varied depot placements, clustered and mixed customer layouts, several demand regimes and route-length classes — each solved to *proven* optimality by branch-cut-and-price. It is the benchmark built to expose the distribution overfitting of learned CVRP solvers, and reporting on it yields an *absolute* optimality gap rather than a gap to a heuristic reference. We run the $N = 100$ checkpoint unchanged, in `float16`, at $T = 10^5$ over the full set in a single GPU batch. LG-SA reaches a mean optimality gap of 3.78% (median 3.35%, three quarters of instances within 5%, the easiest solved to optimality) in 29 min — only 1.5 points above its 2.31% in-distribution gap to HGS at the same budget (Table 3). The XML100 figure is thus the price of a one-distribution shift with no retraining.

Table 32 places this against the eight metaheuristics benchmarked on XML100 by Queiroga et al. (2026). Two things

	Method	Best	Mean
Metaheuristics‡	HGS-CVRP	0.001	0.003
	AILS-II	0.002	0.009
	FILO2	0.014	0.041
	FILO	0.018	0.052
	SISRs	0.018	0.069
	KGLS-XXL	0.078	0.158
	LKH-3	0.250	0.479
	OR-Tools	1.006	1.452
Learned	LG-SA (T=10^5)	—	3.78

Table 32: Optimality gap (%) on the 10,000-instance XML100 benchmark (Queiroga et al. 2022), where every instance is solved to proven optimality. ‡Metaheuristic rows are from Queiroga et al. (2026) (Table 4, overall, averaged over instance attributes), each method given 10 runs under a one-minute-per-instance budget; "Best"/"Mean" are the best and mean over those runs. LG-SA is our $N = 100$ checkpoint, run zero-shot in `float16` at $T = 10^5$ over the whole set in one GPU batch; it is a single run, so only a mean is reported.

must be read carefully. First, those solvers are each given a *one-minute wall-clock budget per instance* — about 7 CPU-days for a single pass over the set, against LG-SA's 29 min for the whole batch on one GPU — so this is a comparison of quality at radically different compute, not a like-for-like race. Second, they are all near-optimal (HGS-CVRP, the reference, is within 0.003%), so LG-SA is neither competitive with them on quality nor meant to be; the point is that a lightweight learned policy trained on a different distribution lands a mean 3.78% from proven optima on a structured benchmark it never saw. No learned method in our comparison set (Table 3) reports XML100 numbers at all, precisely because this is the regime in which such methods are expected to break down.

Restart variance. Because the search is stochastic — a random constructive start followed by sampled, Metropolis-filtered proposals — a fixed instance does not have a fixed answer. Figure 26 isolates this by replicating a single $N = 100$ instance 10,000 times, giving every copy an independent random initialization, and running all restarts as one batch at $T = 10^4$ (the whole study takes 177 s on one GPU). The search contracts the spread sharply but does not close it: the coefficient of variation falls from 3.95% across the initial solutions to 1.19% across the final ones (final cost 15.20 ± 0.18, best 14.68, worst 15.89), so a gap of 1.21 routing units — about 8% — still separates the luckiest restart from the unluckiest. This makes restarts a cheap quality lever with steeply diminishing returns: a single run lands 3.50% above the best of all 10,000, best-of-10 already recovers most of the gap (mean 14.93, -1.8%), best-of-100 reaches 14.80 and best-of-1,000 only 14.74. The effect is one reason the paper's batched evaluation reports a mean over 10,000 instances rather than any single run. This is a single-instance illustration — the exact figures are specific to it — but the qualitative picture (a wide start distribution compressed to a narrow, non-degenerate final one) is what the batched protocol averages over.

Limitations. Three caveats bound what this section supports. First, the dimension sweep has no blind-SA control at the large sizes: it shows that the guided search remains usable up to $N = 10{,}000$, not that the learned policy still contributes there, and the matched guided-against-blind comparison of Table 3 exists only up to $N = 100$. Second, it averages 3 runs per cell, and only the sizes without a pre-generated test set (200, 2,000, 5,000, 10,000) draw a fresh instance per run — at $N \in \{100, 500, 1{,}000\}$ the three runs share one instance and differ only in the random constructive initialization, so the spread there understates instance-to-instance variability. Third, no reference solution is available beyond $N = 100$, so quality at large sizes is reported only as improvement over the constructive initialization, which is a weak yardstick: a large relative improvement over a random-order solution does not establish proximity to the optimum.

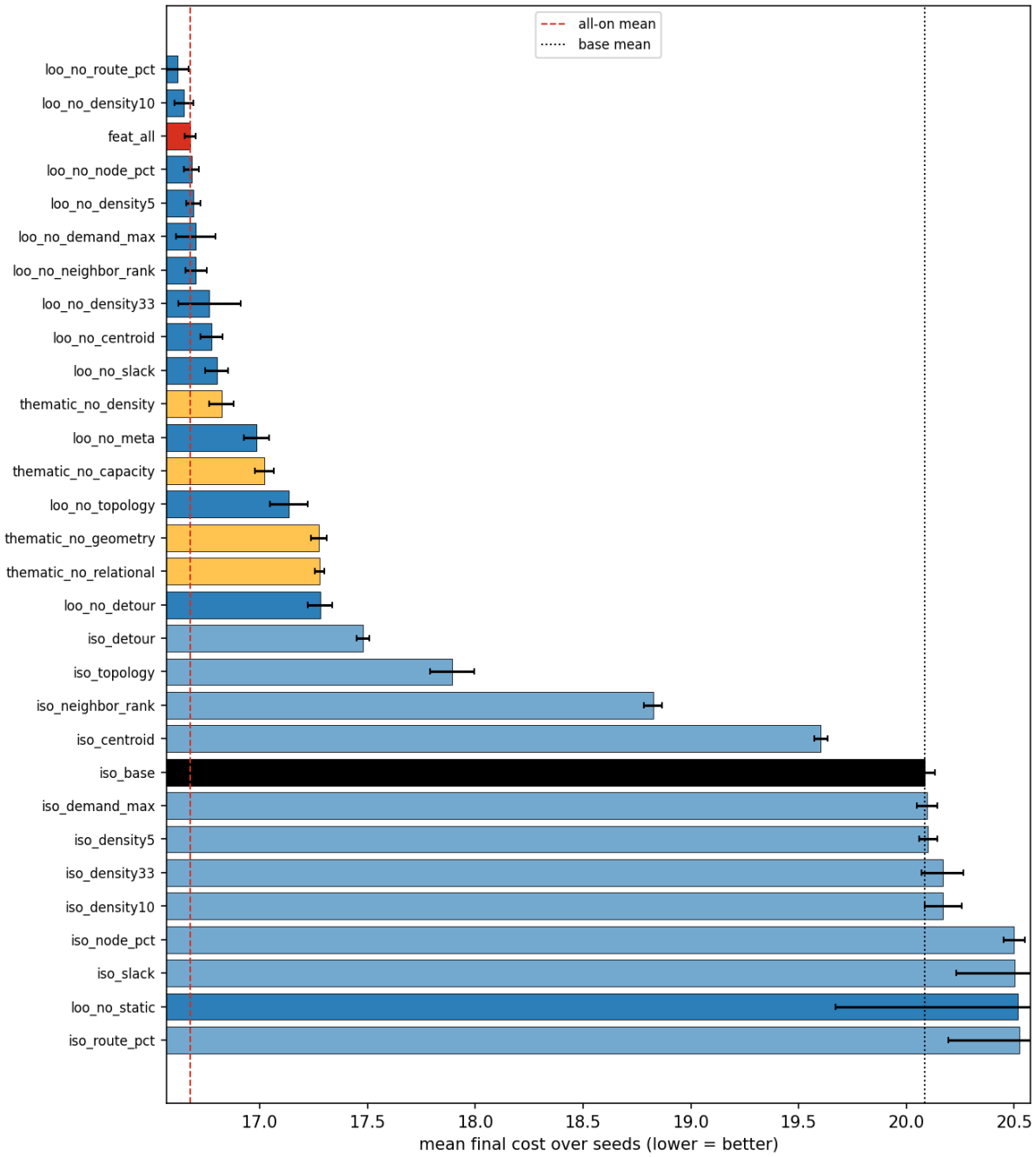

Figure 8: All 30 ablation configurations ranked by mean final cost. Leave-one-out configurations (dark blue) cluster tightly near the full-set reference (red dashed); isolation configurations (light blue) sit near the static+meta base (dotted).

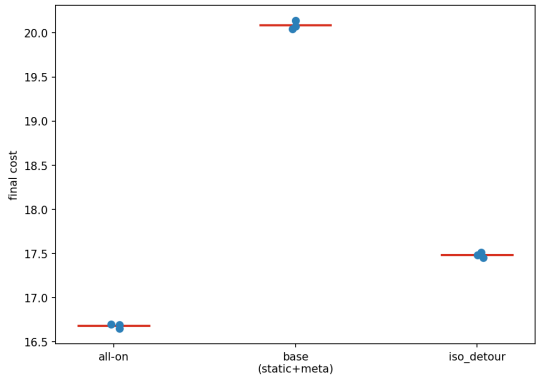

Figure 9: Per-seed final cost for the key reference configurations (red bars = means). Seed variance is tiny except for the no-static configuration, which inflates the pooled noise floor of the 3-seed sweep.

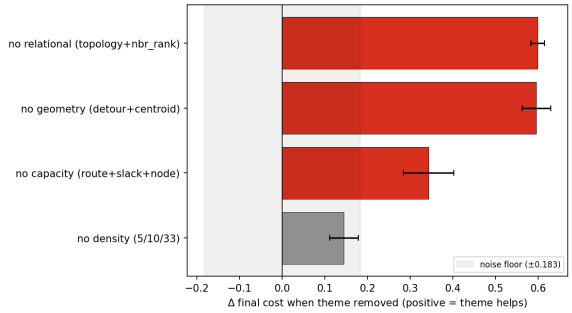

Figure 10: Thematic ablation (Table 10): removing the relational or geometry cluster costs ≈ 0.6 routing units each.

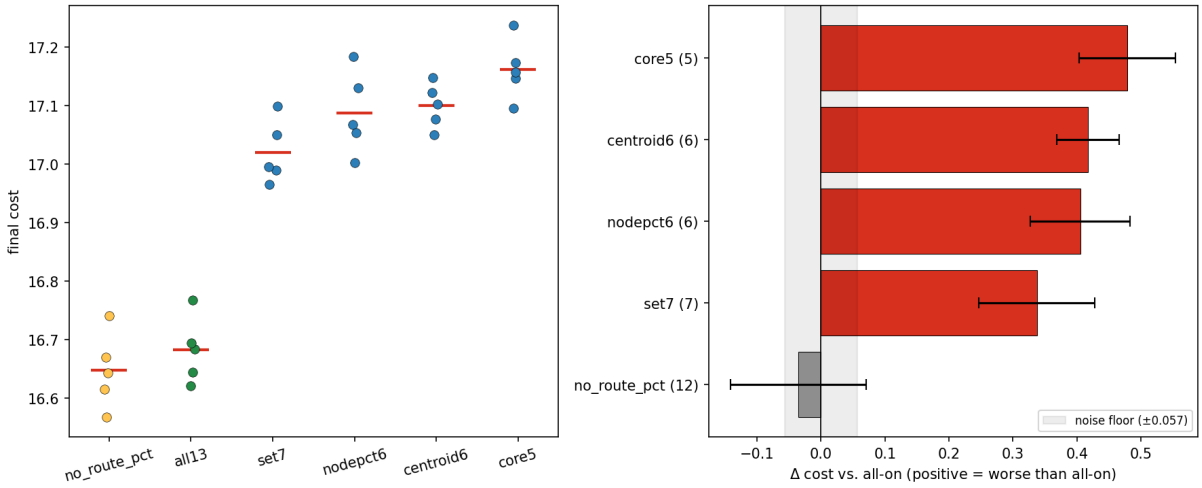

Figure 11: Best-set validation, per-seed view. Left: per-seed final cost — the 12-group and full-set clusters overlap and sit cleanly below every pruned set. Right: seed-paired Δ vs. the full set — the pruned sets (red) clear the noise floor by a wide margin; the 12-group set (green) sits below zero.

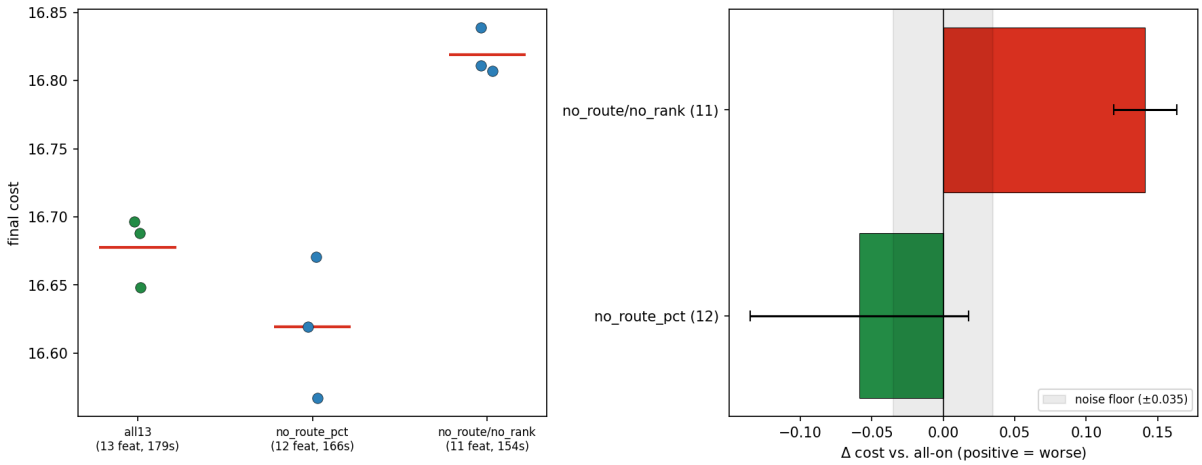

Figure 12: Targeted refinements (Table 11). Left: per-seed final cost — dropping the load ratio overlaps the full set, while additionally dropping the neighbor ranks lifts the whole cluster. Right: seed-paired Δ vs. the full set — the load-ratio removal (green) sits inside the noise floor, the neighbor-rank removal (red) clears it decisively.

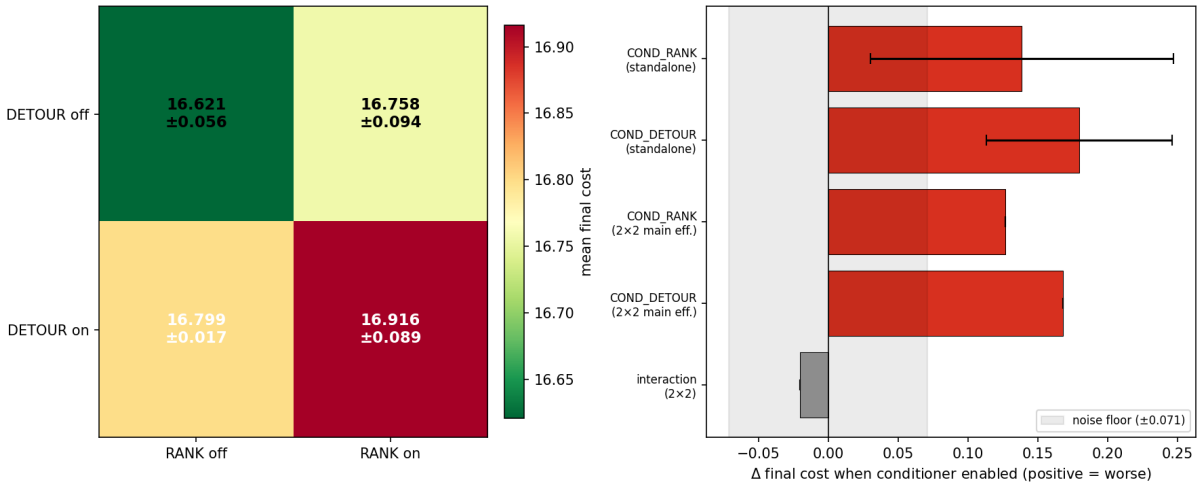

Figure 13: Conditional pair descriptors. Left: 2×2set of configurations of mean final cost — the 'both-off' configuration (plain retained set) is best, 'both-on' is worst. Right: standalone deltas and 2×2 main effects — both descriptors sit clearly beyond the noise floor on the harmful side; the interaction is within it.

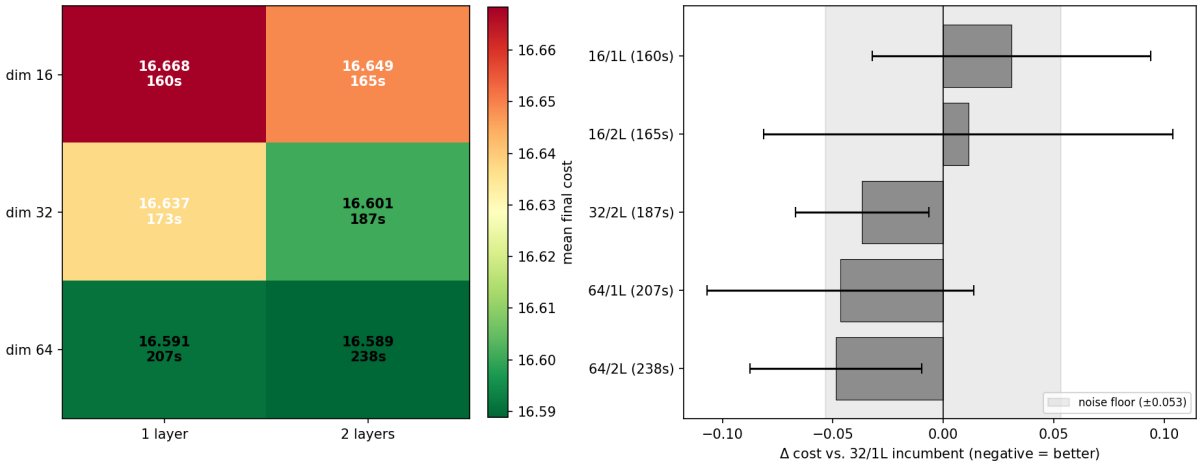

Figure 14: Actor architecture set of configurations (Table 13). Left: mean final cost and wall-clock per configuration (green = better). Right: seed-paired Δ against the 32×1 reference — every configuration falls inside the shaded noise band, so no architectural change is a significant win.

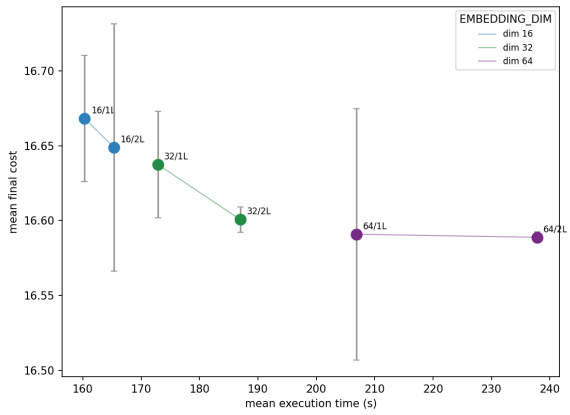

Figure 15: Cost against compute across the six architecture configurations, colored by embedding width. Quality improves only marginally along a steeply rising compute axis — the hallmark of a capacity plateau.

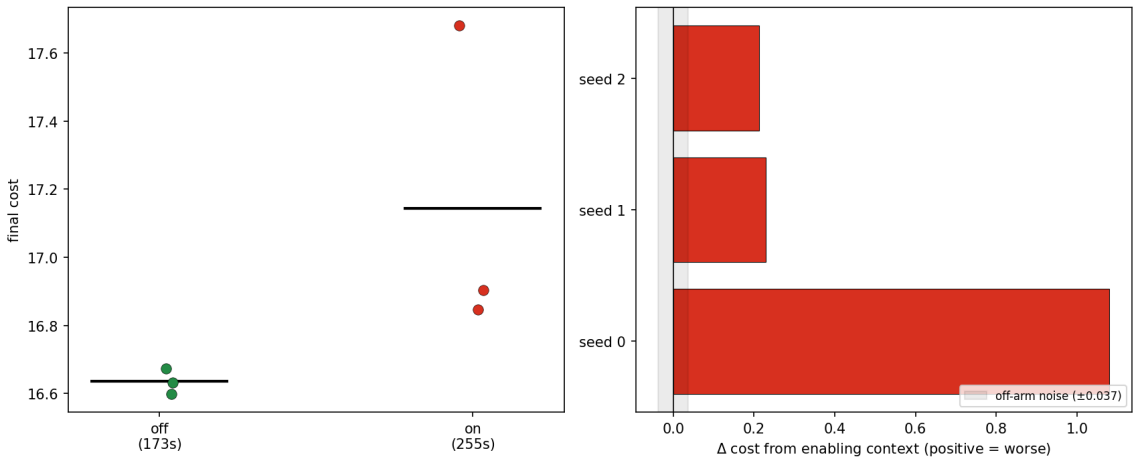

Figure 16: Global-context toggle. Left: per-seed final cost (black bars = means); the context-on arm is both worse and far more variable. Right: per-seed Δ from enabling context — every seed is positive (worse), one dramatically so, so the degradation is not a variance artefact.

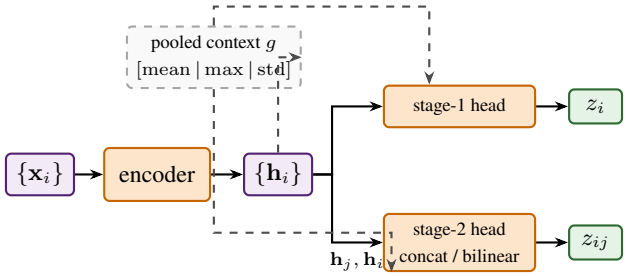

Figure 17: Shared-encoder actor variant. Each node is encoded once into an embedding $\mathbf{h}_i$; two lightweight heads then score the two stages over the shared embeddings, either by concatenation or as a bilinear compatibility between the selected node and each candidate. An optional pooled summary g of all node embeddings (dashed) is appended to both heads' inputs, giving every score access to instance-level context.

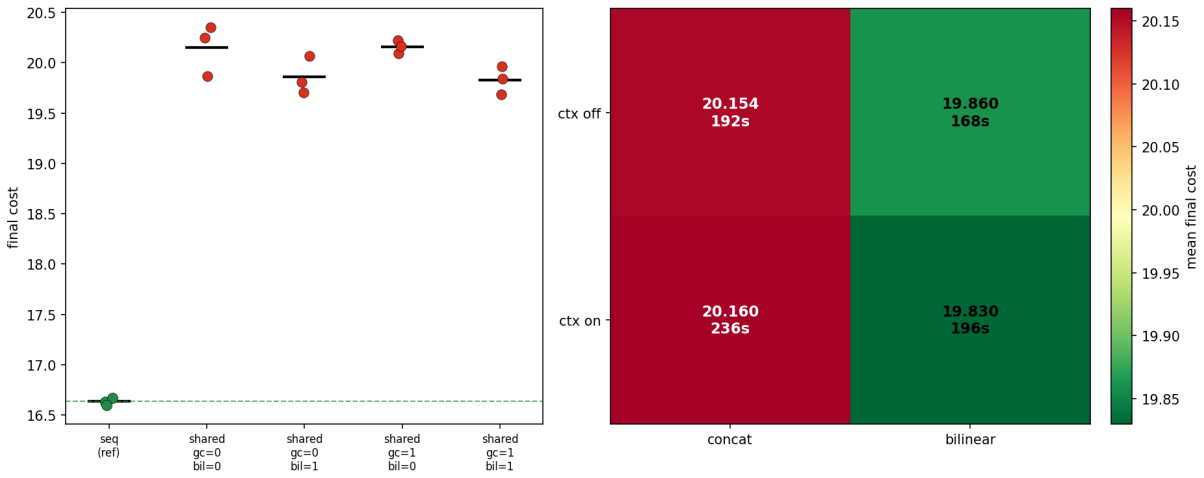

Figure 18: Shared encoder against independent scorers (Table 14). Left: per-seed final cost, reference actor (green) against the four shared configurations (red); the dashed line marks the reference mean, and every shared seed sits far above it. Right: the shared 2×2 set of configurations — the bilinear head beats concatenation and context is roughly neutral, but all four configurations are ≈ 3 units worse than the reference.

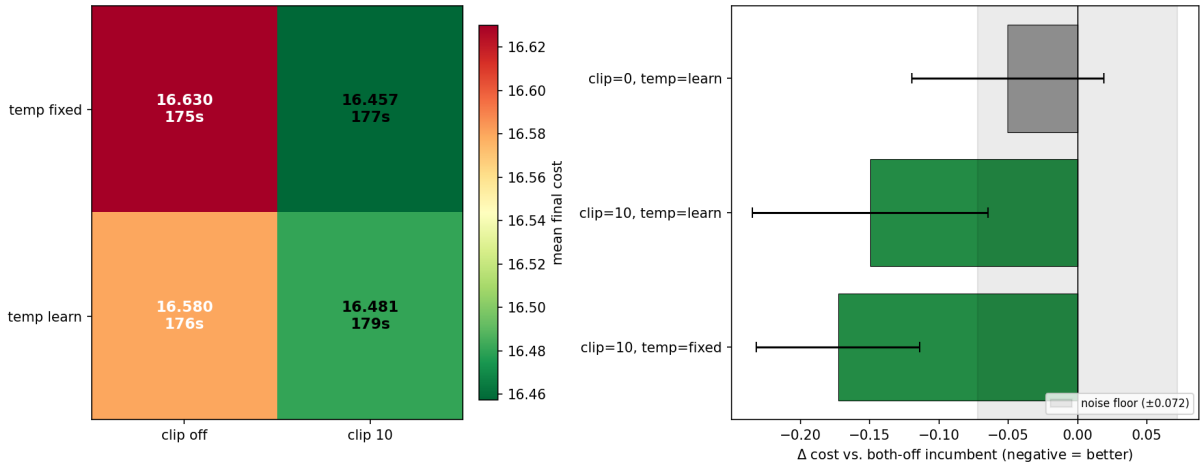

Figure 19: Distribution shaping (Table 15). Left: 2×2 set of configurations of mean final cost and time (green = better); the clipping-on, fixed-temperature configuration is the best. Right: seed-paired Δ against the both-off reference — only logit clipping escapes the noise band, and stacking the learnable temperature on top of it does not help.

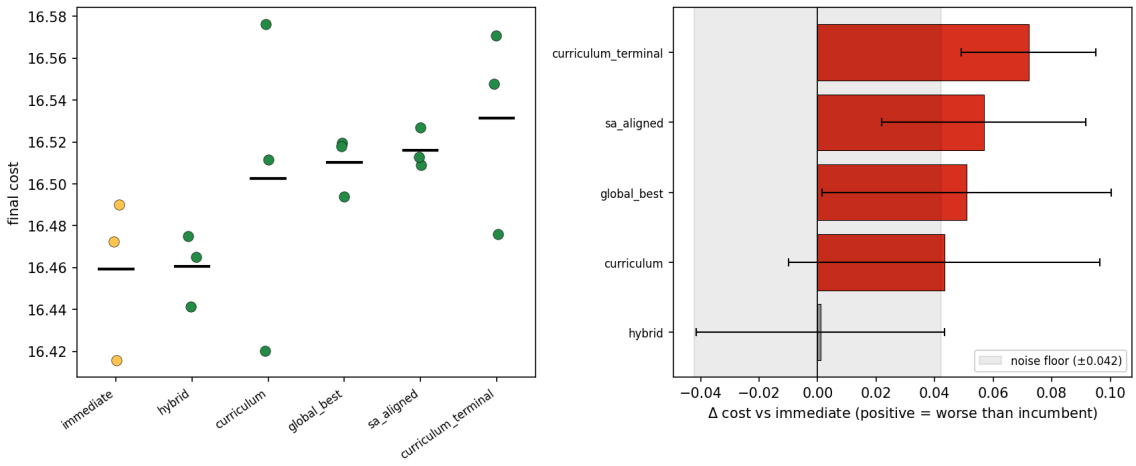

Figure 20: Zoom on the stable reward cluster. Left: per-seed final cost (black bars = means); the arms overlap heavily. Right: seed-paired Δ against the reference immediate reward — the fixed mixture sits inside the noise band (a tie), the remaining dense variants are marginally but consistently above zero.

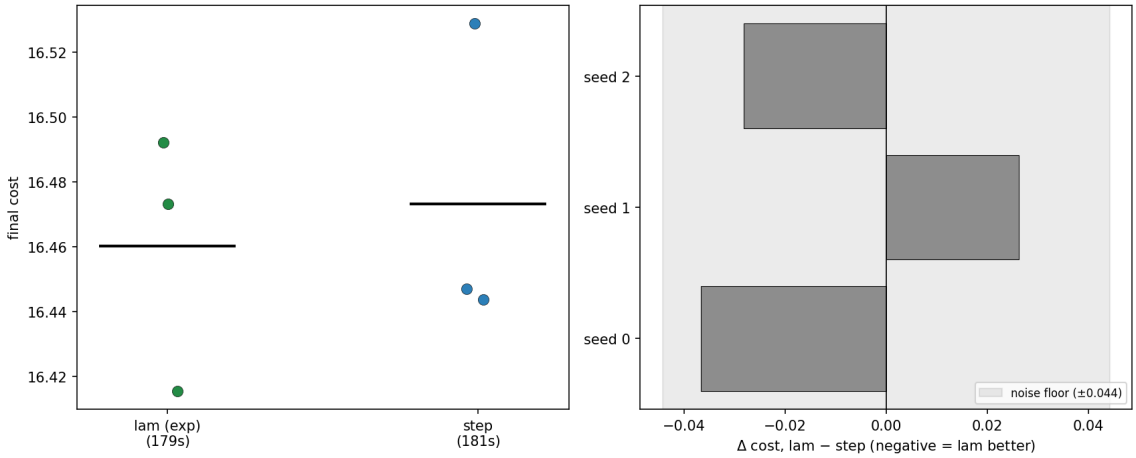

Figure 21: Cooling-schedule shape comparison. Left: per-seed final cost for the two schedules (black bars = means); the clusters overlap heavily. Right: per-seed paired Δ (geometric − step) — every bar lies inside the noise band and the sign is not consistent, the signature of a tie rather than an effect.

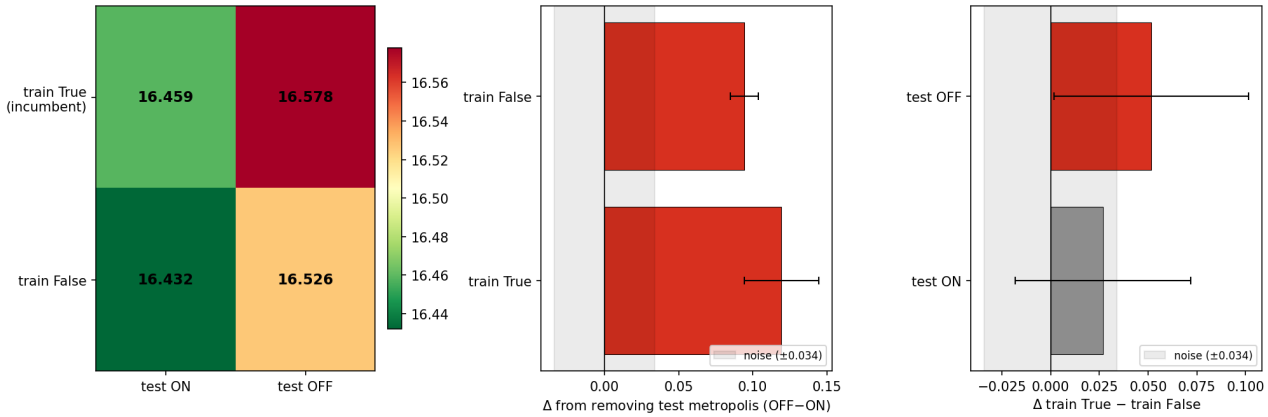

Figure 22: Metropolis train × test cross (Table 18). Left: 2×2 heatmap of mean final cost (greener = better). Center: cost of removing test-time Metropolis (off − on) at each training setting — both bars are well past the noise band, the dominant effect. Right: training-flag effect (on − off) at each inference — the test-on bar sits inside the band (a tie), test-off just clears it.

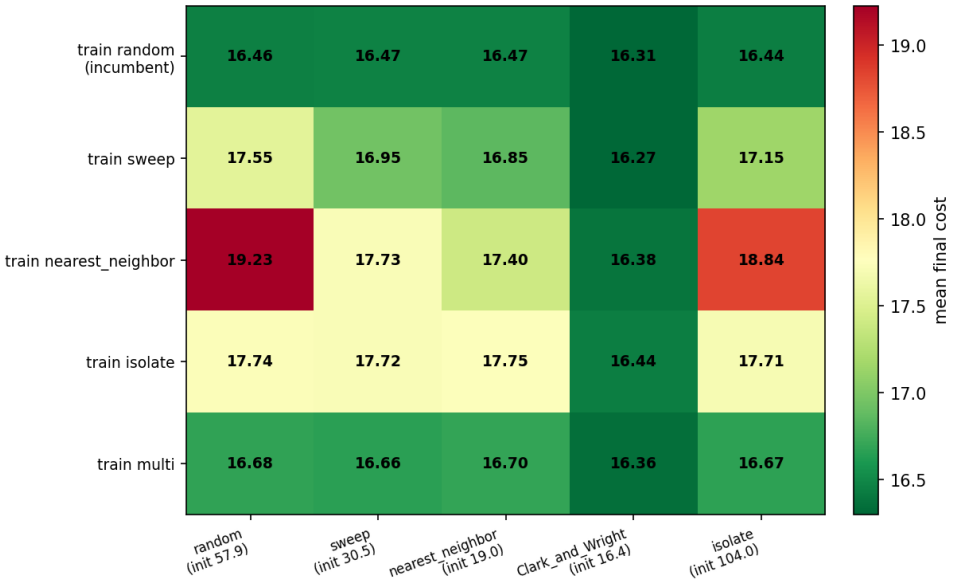

Figure 23: Initialization cross (Table 19): rows = training construction, columns = test-time construction, greener = lower final cost. The random-trained and mixed-trained rows are uniformly good; the three structured-training rows are poor and only recover in the Clarke–Wright column, where the construction itself already did the work.

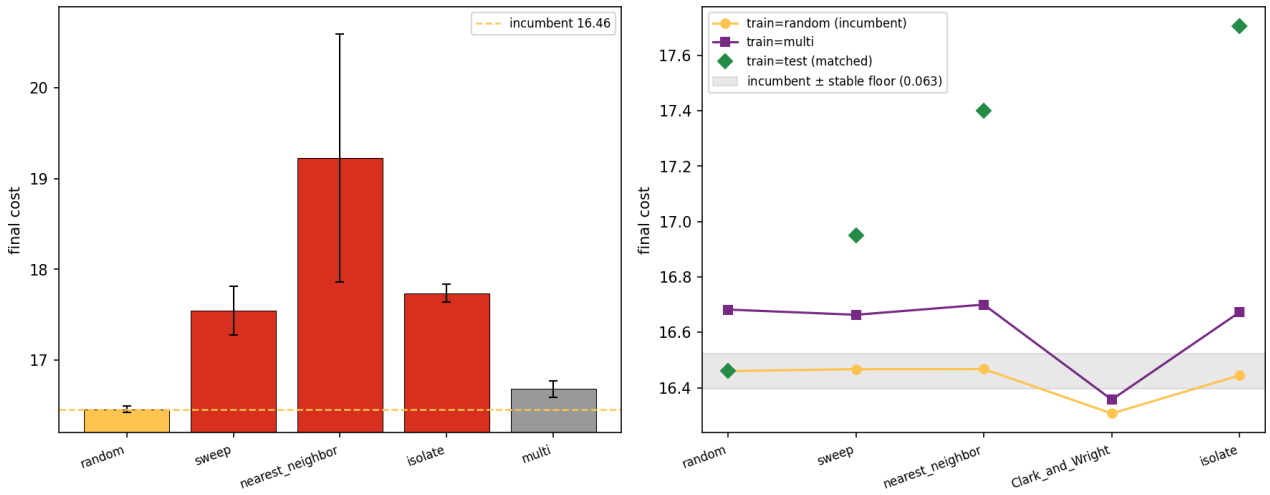

Figure 24: Left: final cost by training construction at the standard random-start inference — random (gold) is far below the rest, with mixed (gray) a close second. Right: final cost of selected actors across the five test constructions: the random-trained actor (gold) is nearly flat and mostly inside the stable-floor band, the mixed actor (purple) sits uniformly a touch above it, and the matched train/test diagonal (green diamonds) climbs away — ruling out a train/test-matching explanation for the training-construction effect.

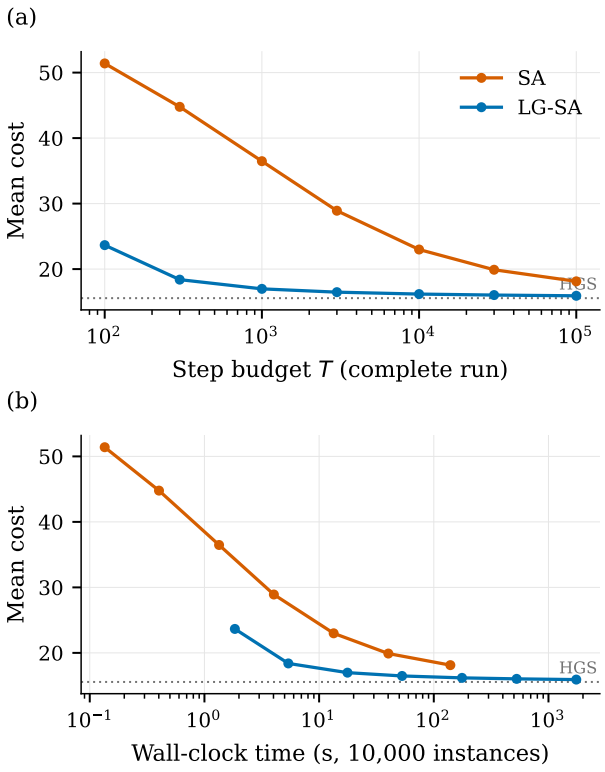

Figure 25: Anytime frontier at $N = 100$ on the 10,000-instance test set, `float16`, identical initial solutions (57.88). Each marker is one complete run whose cooling schedule is compressed to that budget, against (a) the step budget and (b) the resulting wall-clock time; the dotted line is the HGS reference of Table 3. LG-SA's cheapest run (10^2 steps, 1.8 s) is why its curve starts later in (b).

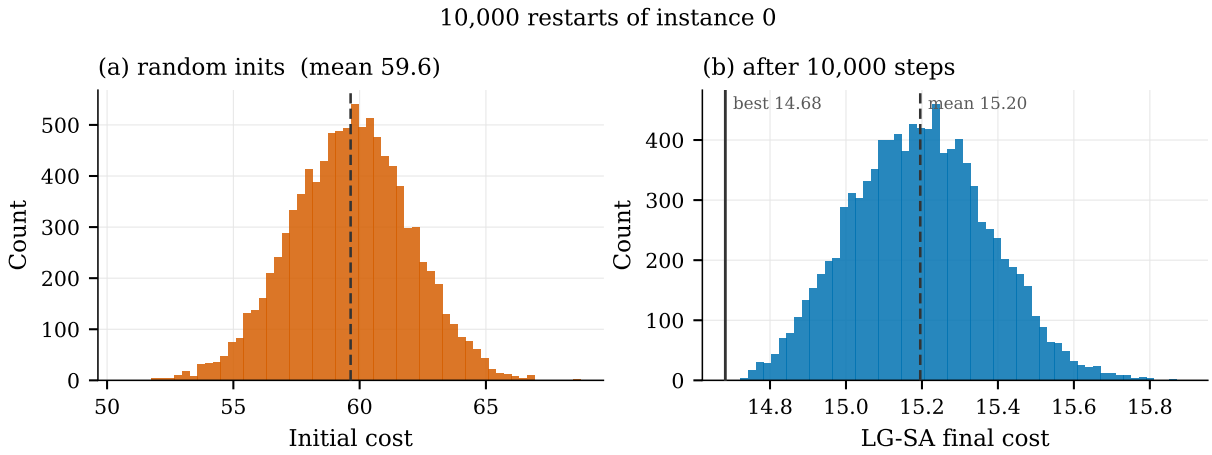

Figure 26: Restart variance on one fixed $N = 100$ instance, `float16`, 10,000 independent random initializations run as a single batch at $T = 10^4$. (a) the constructive starting solutions and (b) the LG-SA final solutions; note the $\approx 15\times$ narrower x-range in (b). Dashed lines mark the means, the solid line in (b) the best restart. The search compresses a CV of 3.95% to 1.19% without collapsing to a single solution.